

Federated neural nonparametric point processes

Hui Chen ^a, Xuhui Fan ^a, Hengyu Liu ^b, Yaqiong Li ^c, Zhilin Zhao ^a,
Feng Zhou ^d, Christopher John Quinn ^e, Longbing Cao ^{a,*}

^a Macquarie University, Sydney, Australia

^b Aalborg University, Aalborg, Denmark

^c Australian Federal Government, Sydney, Australia

^d Renmin University of China, Beijing, China

^e Iowa State University, Ames, USA

ARTICLE INFO

Keywords:

Federated learning
Temporal point processes
Federated neural point processes
Neural embedding
Gaussian processes

ABSTRACT

Temporal point processes (TPPs) are effective for modeling event occurrences over time but struggle with sparse and uncertain events in federated systems, where privacy is a major concern. To address this, we propose *FedPP*, a federated neural nonparametric point process model. FedPP integrates neural embeddings into sigmoidal Gaussian Cox processes (SGCPs) on the client side. SGCPs is a flexible and expressive class of TPPs, allowing FedPP to generate highly flexible intensity functions that capture client-specific event dynamics and uncertainties while efficiently summarizing historical records. For global aggregation, FedPP introduces a divergence-based mechanism to communicate the distributions of kernel hyperparameters in SGCPs between the server and clients, while keeping client-specific parameters local to ensure privacy and personalization. FedPP effectively captures event uncertainty and sparsity. Extensive experiments demonstrate its superior performance in federated settings, showing global aggregation with the KL divergence and the Wasserstein distance.

1. Introduction

Temporal point processes (TPPs) [1–4] have shown great promise in modeling the occurrences of events over a specific observation period. TPPs have been successfully applied to various domains, including neuroscience [5,6] to study neural firing patterns, healthcare [7,8] for predicting medical incidents or patient admissions, finance [9,10] for market event analysis, and mobile app-based ride-hailing [11,12] for predicting transport demand. These applications highlight the flexibility and broad utility of TPPs in modeling event-driven data across diverse fields.

Despite their utility, traditional TPPs methods face significant challenges when dealing with event data that is sparse, uncertain, and distributed in federated systems. For instance, in the context of sequential shopping behaviors, event sequences are often private, and aggregating all these sequences may lead to privacy risks. Moreover, the current TPPs models are usually centralized but not client-specific, making them unable to capture the sparsity and uncertainty of events in each client. In other words, privacy-preserving methods capable of modeling the inherent complexity of event data are needed in practical situations.

Federated learning (FL) [13–16] offers a promising solution for addressing privacy concerns by decentralizing model training across clients. However, to the best of our knowledge, no existing work has explored federated learning in the context of TPPs,

* Corresponding author.

E-mail address: longbing.cao@mq.edu.au (L. Cao).

particularly with challenges related to event sparsity and uncertainty. This raises key research questions: *How can we model client-specific event sequences while preserving privacy? How can we aggregate knowledge across clients without centralizing sensitive event data?*

In this paper, we address these questions by leveraging sigmoidal Gaussian Cox processes (SGCPs) [17–19], which have been successfully used as a powerful TPP model. Following typical TPP models, SGCPs model an intensity function over the observation period. In particular, the intensities of events are learned using a Gaussian process, which can capture complex temporal patterns, while a sigmoidal transformation ensures valid non-negative intensity values. However, SGCPs are limited by the expressiveness of their kernel functions, particularly in capturing the nuances of event sparsity and uncertainty across clients and in effectively using historical information.

To overcome these limitations, we propose FedPP, a Federated neural nonparametric Point Process model. FedPP integrates neural embedding [11,20,21] into SGCP kernels to improve their flexibility and expressiveness and decentralizes the training process across clients to preserve privacy. On the *client side*, the integrated neural embeddings may effectively summarize historical event information and generate flexible client-specific intensity functions. On the *server side*, FedPP develops a novel divergence-guided global aggregation mechanism to communicate distributions of neural embeddings rather than individual parameters. This ensures common event information and uncertainties are shared in a privacy-preserving manner. By aligning the global distribution with client-specific distributions, FedPP personalizes the model for each client while maintaining a strong global performance. This structure effectively addresses both the privacy and modeling challenges in federating TPPs.

In summary, we make the following contributions:

- We propose FedPP, the first federated version of Temporal Point Process models, which bridges the gap between TPPs and federated learning, to address the challenges of event sparsity, uncertainty, and privacy concerns.
- We introduce a novel integration of neural embedding methods into the kernels of SGCPs, which significantly improves their expressiveness and enabling the effective utilization of historical information.
- We develop a divergence-guided global aggregation mechanism that shares distributions of neural embeddings between the server and clients, to enable robust global modeling while preserving client-specific privacy.

Our method demonstrates superior performance on benchmark datasets, effectively capturing event sparsity and uncertainty in federated settings without compromising privacy.

2. Background

TPPs provide a fundamental framework for modeling event occurrences over time with a broad range of applications in fields such as healthcare, finance, and social networks. Several advanced methods have been developed to improve the expressiveness of TPPs, including SGCPs for nonparametric modeling and neural embedding methods for capturing historical event dependencies. In this section, we provide an overview of TPPs, introduce SGCPs as a key nonparametric approach, and discuss neural embedding techniques that leverage historical event data to improve event prediction.

2.1. Temporal point processes (TPPs)

[22] provides a probabilistic framework for modeling a set of random events over a time period. In TPPs, the rate of event occurrences is characterized by an intensity function $\lambda(t)$:

$$\lambda(t) = \lim_{\Delta t \rightarrow 0^+} \frac{\mathbb{E}[\mathcal{N}(t + \Delta t) - \mathcal{N}(t)]}{\Delta t}, \quad (1)$$

where $\mathcal{N}(t)$ denotes the number of events occurring up to time t . The intensity function $\lambda(t)$ is a non-negative function of time, indicating the likelihood of event occurrence at time t . A larger value of $\lambda(t)$ imply a higher chance of event occurrence. Given a set of N observed event times $\mathbf{t} = \{t_1, \dots, t_N\}$ with $t_i \in [0, T]$, the likelihood of observing this sequence in a point process is:

$$\mathcal{L}(\lambda(t) | \{t_1, \dots, t_N\}) = e^{-\int_0^T \lambda(t) dt} \prod_{i=1}^N \lambda(t_i). \quad (2)$$

2.2. Sigmoidal Gaussian Cox processes (SGCPs)

SGCPs [17–19] is a powerful nonparametric framework for learning event data. In SGCPs, the intensity function $\lambda(t)$ is modeled as a sigmoidal transformation of a random function drawn from a Gaussian process, expressed as $\lambda(t) = m \cdot \sigma(f(t))$, where $f(\cdot) \sim \mathcal{GP}(\nu(\cdot), \kappa_{\cdot, \cdot; \mathbf{w}})$, and $\sigma(\cdot) = 1/(1 + \exp(-\cdot))$ ensures the intensity is within $(0, 1)$. Here, m is a scaling parameter, $f(t)$ is the random function, $\nu(t)$ is the mean function, and $\kappa_{t, t'; \mathbf{w}}$ is the kernel function with hyperparameters $\mathbf{w}$. SGCPs are Bayesian nonparametric models capable of representing highly flexible intensity functions. However, traditional SGCPs rely on standard kernels like the RBF kernel, which may be inefficient to generate highly flexible intensity functions. Further, it might be difficult to directly use the rich historical information to predict future events for SGCPs.

2.3. Neural embedding methods for event data

For a given event t_i , its history, denoted as $\mathcal{H}_{t_i} = \{t_{i'} : t_{i'} < t_i\}$, can significantly influence the occurrence of the next event. Neural embedding methods [11,20,21] encode this history into a vector $\mathbf{h}_{i-1}$, referred to as the historical embedding. This embedding can be used to parameterize the conditional distribution of the next event time t_{i+1} :

$$t_{i+1} \sim P_\theta(t_{i+1}|\mathbf{h}_i), \quad \text{where } \mathbf{h}_i = f_{\text{update}}(\mathbf{h}_{i-1}, \mathbf{e}_i). \quad (3)$$

Here, f_{update} represents a recurrent update function such as RNN, LSTM, or an attention-based layer, and $\mathbf{e}_i$ is the embedding of event t_i [23]. Neural embedding methods are effective in summarizing historical event information. However, many of these methods adopt an autoregressive strategy to predict event times based on previous events rather than learn the intensity function directly from the TPPs perspective, which may lead to a lack of a clear statistical understanding of event dynamics over time.

3. The FedPP method

In this section, we introduce the Federated neural nonparametric Point Process model (FedPP). FedPP is designed to address the challenges of modeling event sparsity and uncertainty in federated systems while preserving data privacy. The main idea of FedPP is to integrate neural embeddings with SGCPs on the client side to model client-specific event dynamics and uncertainties while effectively summarizing historical information. It also introduces a divergence-based mechanism for sharing kernel hyperparameters between the server and clients, while keeping client-specific parameters local to preserve privacy and support personalization. This section is structured as follows:

- **Client-side modeling** (Section 3.1): We describe how each client uses neural SGCPs to model its event sequence, including the use of neural embeddings to summarize historical information, and sparse Gaussian processes to reduce computational complexity.
- **Server-side aggregation** (Section 3.2): When targeting on aggregating parameters' distributions, we further introduce a divergence-guided global aggregation mechanism that updates global distribution based on the clients' variational distributions.
- **Bi-level optimization** (Section 3.3): We formalize the overall optimization framework for FedPP as a bi-level problem, combining local and global updates.
- **Detailed algorithm steps** (Section 3.4): We present the FedPP algorithm in a practical federated learning setting, outlining the three key steps—computing expected log-likelihood, applying mean-field variational inference, and optimizing the local objective function using first-order methods.

3.1. Client-side modeling: Neural SGCPs

Each client in the FL framework models its observed event sequence $\mathbf{t}_c = \{t_{c,i}\}_{i=1}^{n_c}$, where n_c is the number of events in client c , using a client-specific SGCP. The intensity function $\lambda_c(t)$ is expressed as:

$$\lambda_c(t) = m_c \cdot \sigma(f_c(t)), \quad f_c(t) \sim \mathcal{GP}(v_c, \kappa_{\cdot, \cdot; \mathbf{w}_c}), \quad (4)$$

where $\mathcal{GP}(v_c, \kappa_{\cdot, \cdot; \mathbf{w}_c})$ is a Gaussian process with client-specific mean v_c and kernel $\kappa_{\cdot, \cdot; \mathbf{w}_c}$. In our neural SGCPs for each client c , neural embedding methods are used to encode the historical information of events and improve the expressiveness of the intensity function.

Neural Embedding. For each event t_i , we encode its history $\mathcal{H}_{t_i} = \{t_{i'} : t_{i'} < t_i\}$ using a neural embedding method, by following Eq. (3). We propose that the resulting embedding $\mathbf{h}_i$ can be used to compute the kernel values between two time points t_i and t_j as:

$$\kappa_{t_i, t_j; [\tilde{\mathbf{w}}; r; l]} = r \cdot \exp\left(-\frac{\|\mathbf{h}_i - \mathbf{h}_j\|^2}{2l^2}\right), \quad (5)$$

where $\tilde{\mathbf{w}}$ refers to the parameters of the neural network forming the historical embedding, r is the scaling parameter, and l is the length-scale parameter for the RBF kernel. By incorporating neural embeddings into the kernel of the SGCPs, we efficiently summarize historical event information and generate a more expressive intensity function. This addresses the traditional kernel limitations in SGCPs.

Prior and Variational Distributions. FedPP sets the prior distribution for the parameter vector $\mathbf{w} = [\tilde{\mathbf{w}}; \log r; \log l]$, which is a combination of the neural network parameters $\tilde{\mathbf{w}}$ and the RBF kernel parameters $\log r$ and $\log l$, as an isotropic multivariate Gaussian distribution:

$$p_\theta(\mathbf{w}_c) = \mathcal{N}(\mathbf{w}_c; \boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)), \quad (6)$$

where $\theta = [\boldsymbol{\mu}; \boldsymbol{\sigma}^2]$, with $\boldsymbol{\mu} \in \mathbb{R}^{(M_{\mathbf{w}}+2) \times 1}$ and $\boldsymbol{\sigma}^2 \in [\mathbb{R}^+]^{(M_{\mathbf{w}}+2) \times 1}$. This prior distribution is shared across all clients for their respective parameter weights $\mathbf{w}_c$. Each client c has its own variational distribution, denoted as $q_{\Phi_c}(\mathbf{w}_c) = \mathcal{N}(\mathbf{w}_c; \mathbf{r}_c, \text{diag}(\boldsymbol{\delta}_c^2))$, where the variational parameters $\Phi_c = [\mathbf{r}_c; \boldsymbol{\delta}_c^2]$ are client-specific, with $\mathbf{r}_c \in \mathbb{R}^{(M_{\mathbf{w}}+2) \times 1}$ and $\boldsymbol{\delta}_c^2 \in [\mathbb{R}^+]^{(M_{\mathbf{w}}+2) \times 1}$.

Since the data on each client is typically non-independent and identically distributed (non-IID) [24], a global model may struggle to generalize effectively across all clients. This represents a key challenge in FL, as heterogeneous data distributions can significantly impact model performance. In our approach, however, each client has its own personalized variational distribution, allowing the model to better adapt to the local data distribution of each client. This personalization effectively addresses the challenges posed by

data heterogeneity. As a result, our method performs robustly in federated systems with heterogeneous data, enabling each client to learn a model tailored to its local environment while still contributing to the global model.

Sparse Gaussian Processes. A sparse Gaussian process is employed for each client to reduce the computational cost of neural SGCPs from $\mathcal{O}(n_c^3)$ to $\mathcal{O}(n_c M^2)$, where M is the number of inducing points $\mathbf{u}_c \in \mathbb{R}^{M \times 1}$. By fixing the inducing locations $\mathbf{z} \in \mathbb{R}^{M \times 1}$ as constants that are equally distributed across the time period for all clients, the generative process of the neural SGCP for client c can be approximated as follows:

$$\mathbf{w}_c \sim p_{\theta}(\mathbf{w}_c), \quad \mathbf{u}_c \sim \mathcal{N}(\mathbf{u}_c; \nu_c, \mathbf{K}_{\mathbf{z}, \mathbf{z}; \mathbf{w}_c}), \quad (7)$$

$$\tilde{f}_c(\cdot) \sim \mathcal{N}(\tilde{f}_c(\cdot); \nu_c + \mathbf{K}_{\cdot, \mathbf{z}; \mathbf{w}_c} \mathbf{K}_{\mathbf{z}, \mathbf{z}; \mathbf{w}_c}^{-1} (\mathbf{u}_c - \nu_c), \mathbf{K}_{\cdot, \cdot; \mathbf{w}_c} - \mathbf{K}_{\cdot, \mathbf{z}; \mathbf{w}_c} \mathbf{K}_{\mathbf{z}, \mathbf{z}; \mathbf{w}_c}^{-1} \mathbf{K}_{\mathbf{z}, \cdot; \mathbf{w}_c}), \quad (8)$$

$$\mathbf{t}_c \sim \text{PoissonProcess}(m_c \cdot \sigma(\tilde{f}_c(\cdot))). \quad (9)$$

Eq. (7) describes the prior distribution of the inducing points $\mathbf{u}_c$, while Eq. (8) represents the prior distribution of the approximated function $\tilde{f}_c(\cdot)$. Finally, Eq. (9) describes the Poisson process with the intensity function $m_c \cdot \sigma(\tilde{f}_c(\cdot))$. These approximations $\{\tilde{f}_c(\cdot)\}_{c=1}^C$ are direct results of the sparse Gaussian process method.

Scalability in Highly Populated Federated Systems. In our proposed approach, we assume that each client has its own independent set of inducing points for sparse Gaussian process modeling. This design ensures that the computational overhead scales linearly with the number of clients. Specifically, the computational complexity for each client is determined primarily by the size of its inducing points, which remains constant regardless of the total number of clients. As a result, the overall computational cost increases linearly as the number of clients grows. This linear scalability is a key advantage in highly populated federated systems, enabling our method to efficiently handle thousands of clients without encountering exponential computational overhead.

3.2. Server-side aggregation: Divergence-guided global aggregation

In FedPP, the prior distribution $p_{\theta}(\cdot)$, whose parameter θ is shared among all the clients, is maintained on the server. During each communication round, the server broadcasts the prior distribution $p_{\theta}(\cdot)$ to all participating clients. After each client updates its variational distribution $q_{\phi_c}(\mathbf{w}_c)$ independently, the variational parameters ϕ_c are uploaded to the server. The global aggregation is performed by minimizing the sum of divergences between $q_{\phi_c}(\mathbf{w}_c)$ and $p_{\theta}(\mathbf{w}_c)$ with respect to θ , expressed as:

$$\min_{\theta} \sum_{c=1}^C \text{DV} [q_{\phi_c}(\mathbf{w}_c) \parallel p_{\theta}(\mathbf{w}_c)], \quad (10)$$

where $\text{DV}[\cdot \parallel \cdot]$ represents a divergence between two probability distributions. This process finds the optimal prior distribution $p_{\theta^*}(\cdot)$ that is closest to all the variational distributions $\{q_{\phi_c}(\mathbf{w}_c)\}_{c=1}^C$ of clients.

Our proposed divergence-guided global aggregation method extends TPPs. This approach can be applied in any federated learning scenario where probability distributions are communicated between the server and clients. Fig. 1 illustrates how θ is updated for the prior distribution $p_{\theta}(\mathbf{w}_c)$.

To solve Eq. (10), we consider several choices for $\text{DV}[\cdot \parallel \cdot]$, primarily by the KL divergence and the Wasserstein distance. There are several ways to define $\text{DV}[\cdot \parallel \cdot]$, each leading to different global aggregation rules:

(1) KL Divergence. One of the most widely used divergence measures is the KL divergence. The sum of KL divergences:

$$\sum_{c=1}^C \text{KL} [q_{\phi_c}(\mathbf{w}_c) \parallel p_{\theta}(\mathbf{w}_c)] \quad (11)$$

reaches its minimum for $\theta = \{\mu, \sigma^2\}$ when:

$$\mu = \frac{1}{S} \sum_{c \in \mathbb{S}} \mathbf{r}_c, \quad \sigma^2 = \frac{1}{S} \sum_{c \in \mathbb{S}} [\delta_c^2 + \mathbf{r}_c^2 - \mu^2], \quad (12)$$

where $\phi_c = \{\mathbf{r}_c, \delta_c^2\}$ and $\mathbb{S}$ is a random set of clients at size S . Unlike the FedAvg mechanism [13,25], which averages $\{\mathbf{r}_c\}_{c=1}^C$ and $\{\delta_c^2\}_{c=1}^C$ for μ and σ^2 , our FedPP method incorporates the sample variance of $\{\mu_c\}_{c=1}^C$, which is $\sum_{c \in \mathbb{S}} [\mu_c^2 - \mu^2]/S$. This adjustment is essential because the spatial distribution of $\{\mu_c\}_{c=1}^C$ can have non-negligible variance.

(2) Wasserstein Distance. Compared to the KL divergence, the Wasserstein distance is more robust, especially when the supports of two distributions differ. The sum of Wasserstein distances:

$$\sum_{c=1}^C W_2 [q_{\phi_c}(\mathbf{w}_c) \parallel p_{\theta}(\mathbf{w}_c)] \quad (13)$$

reaches its minimum for $\theta = \{\mu, \sigma^2\}$ when:

$$\mu = \frac{1}{S} \sum_{c \in \mathbb{S}} \mathbf{r}_c, \quad \sigma = \frac{1}{S} \sum_{c \in \mathbb{S}} \delta_c. \quad (14)$$

In this case, the Wasserstein distance computes the average of the standard deviations of clients, yielding a more conservative estimate compared to FedAvg and the KL divergence.

(3) Other Divergence Measures. When closed-form solutions for θ or a closed-form expression of the divergence are unavailable, we may use the reparameterization trick to optimize θ . We demonstrate this approach using the maximum mean discrepancy in Section G.

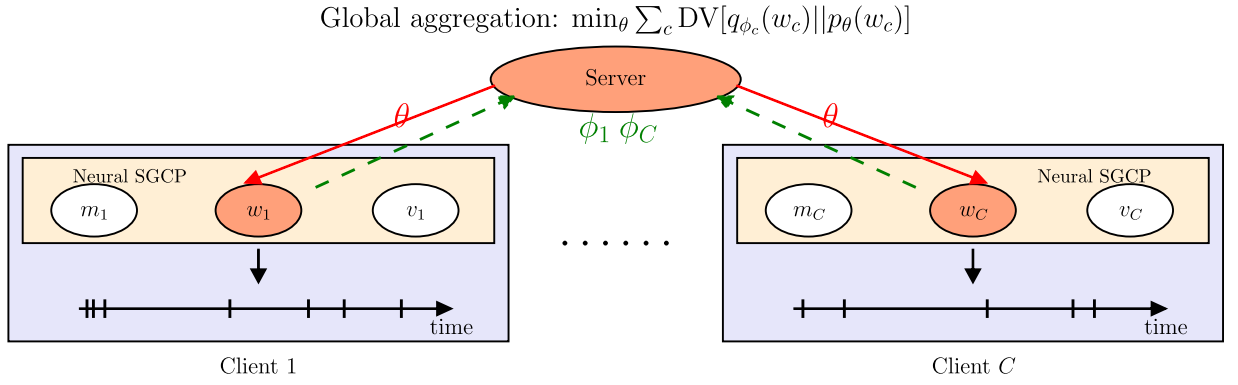


Fig. 1. Federated point process communication between server and clients. The variational distribution $q_{\phi_c}(w_c)$ of clients are updated to the server, while m_c and v_c are kept at the clients. The global aggregation optimizes θ to minimize the sum of divergences between the variational distribution $q_{\phi_c}(w_c)$ of clients and its prior $p_{\theta}(w_c)$. The server then broadcasts the optimized θ value to all the clients.

3.3. Bi-level optimization

The inference process of FedPP is formulated as a bi-level optimization problem. The outer optimization aims to minimize the global objective function $F(\theta)$, which is defined as the average of local objective functions across all clients:

$$\min_{\theta} F(\theta) := \frac{1}{C} \sum_{c=1}^C F_c(\theta), \quad (15)$$

where $F_c(\theta)$ is the local objective function for client c , defined as:

$$F_c(\theta) := \min_{\phi_c, v_c, m_c} \left\{ DV[q_{\phi_c}(w_c) || p_{\theta}(w_c)] - \mathbb{E}_{q_{\phi_c}(w_c)} [\log p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c, v_c)] \right\} \quad (16)$$

In Eq. (15), $F(\theta)$ represents the global objective function, and the local objective function $F_c(\theta)$ in Eq. (16) is minimized at the client level.

For each client, we use a negative evidence lower bound (ELBO)-based objective function that consists of two main components: a reconstruction loss and a divergence term. The reconstruction loss is:

$$-\mathbb{E}_{q_{\phi_c}(w_c)} [\log p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c, v_c)], \quad (17)$$

and the divergence term is:

$$DV[q_{\phi_c}(w_c) || p_{\theta}(w_c)]. \quad (18)$$

These two terms are jointly optimized to update the variational parameters ϕ_c . Note that only the divergence term in Eq. (15) depends on the global parameter θ .

In FedPP, the variational distribution of w_c participates in the communication between the server and clients, as learning w serves as a universal feature extractor across all clients. Meanwhile, the mean functions $\{v_c\}_{c=1}^C$ and the inducing points $\{u_c\}_{c=1}^C$ of GP, and the scaling parameters $\{m_c\}_{c=1}^C$ of SGCP are optimized independently for each client, reflecting the unique characteristics and tasks of each client. This allows FedPP to act as a personalized federated learning framework, enabling client-specific models while maintaining global knowledge sharing.

3.4. The FedPP algorithm

We propose a novel FL algorithm to instantiate FedPP in a practical cross-device FL setting, where a random subset S_j of all clients participate in each communication round. Directly optimizing Eq. (16) is challenging due to the absence of an analytical form for the reconstruction loss. To address this issue, we outline a three-step strategy.

Step 1: Calculating $\mathbb{E}_{q_{\phi_c}(w_c)} [\log p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c, v_c)]$. The likelihood of events for client c , $p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c, v_c)$, is computed by integrating out the approximated function $\tilde{f}_c(t)$ and the inducing points u_c :

$$p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c, v_c) = \int p(\mathbf{t}_c | m_c, \tilde{f}_c(t)) p(\tilde{f}_c(t) | u_c, \mathbf{w}_c, \mathbf{z}, v_c) p(u_c | \mathbf{z}, \mathbf{w}_c) du_c d\tilde{f}_c(t), \quad (19)$$

where $\tilde{f}_c(t)$ is the approximated function and u_c refers to the inducing points.

The expression $p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c, v_c)$ involves integration and sigmoidal operations in the exponential term, which makes direct inference challenging. To overcome this, we adopt the SGCPs model [17–19,26,27] and augment it with latent marked Poisson processes

Π_c , where $m_c \times p_{PG}(\xi|1, 0)$ serves as the intensity function. Additionally, Pólya-Gamma random variables $\{\xi_{c,i}\}_{i=1}^{n_c}$, governed by the Pólya-Gamma distribution $p_{PG}(\xi|1, 0)$, are incorporated. This allows us to express the augmented likelihood as:

$$p(\mathbf{t}_c | \tilde{f}_c(\cdot), m_c) = \int p(\mathbf{t}_c, \Pi_c, \{\xi_{c,i}\}_{i=1}^{n_c} | m_c, \tilde{f}_c(\cdot)) \cdot \prod_{(t_{c,j}, \xi_{c,j}) \sim \Pi_c} \left[e^{g(\xi_{c,j}, -f_c(t_{c,j}))} m_c p_{PG}(\xi_{c,j} | 1, 0) \right] \cdot d\Pi_c d\{\xi_{c,i}\}_{i=1}^{n_c}, \quad (20)$$

where $g(\xi, f) = f/2 - \xi f^2/2 - \log 2$. This results in a closed-form expression for the ELBO.

Step 2: Mean-field Variational Inference for Random Variables $\mathbf{u}_c$, ξ_c , and Π_c . To simplify the inference, we assume independence between the random variables $\mathbf{u}_c$, ξ_c , and Π_c . By using the standard mean-field variational inference method [28,29], we compute the optimal variational distributions as follows:

$$\ln q_{x_i}(x_i) = \mathbb{E}_{q_{\setminus x_i}} [\ln p(D, \{x_i\}_i)] + \text{const}. \quad (21)$$

It is noted that we are using KL-divergence between the variational distribution of $\mathbf{u}_c$, ξ_c , Π_c and their prior distribution here. The divergence $\text{DV}[\cdot \| \cdot]$ is used for the case of $\mathbf{w}_c$ only. That is, we are using separate divergences for $\mathbf{u}_c$, ξ_c , Π_c and $\mathbf{w}_c$. Further details of the calculation for each of $\mathbf{u}_c$, ξ_c , Π_c are provided in Appendix F.

Step 3: Maximizing the Local Objective Function w.r.t. $q_{\Phi_c}(\mathbf{w}_c)$. The local objective function for client c is abstracted as:

$$L_c(\Phi_c) := -\mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)} [\text{DV}[q(\mathbf{u}_c) \| p(\mathbf{u}_c | \mathbf{w}_c, \mathbf{z})]] + \mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)} [\log p(\mathbf{t}_c | \mathbf{w}_c, -)] - \text{DV}[q_{\Phi_c}(\mathbf{w}_c) \| p_{\theta}(\mathbf{w}_c)]. \quad (22)$$

We can leverage the reparameterization trick to compute a closed-form expression for the first two terms. The last term is straightforward to calculate since both $q_{\Phi_c}(\mathbf{w}_c)$ and $p_{\theta}(\mathbf{w}_c)$ follow Gaussian distributions.

Brief Summary of the Local Updates: At the j th communication round, the global distribution $p_{\theta(j)}(\mathbf{w}_c)$ is sent to each client c . During each local epoch, the variational distributions of $\mathbf{u}_c$, ξ_c , and Π_c are updated using Eq. (21). Then, the variational parameters Φ_c are optimized using first-order stochastic gradient descent (SGD):

$$\Phi_c^{(e)} \leftarrow \text{SGD}(-L_c(\Phi_c); B, E, \eta), \quad (23)$$

where B , E , and η represent the mini-batch size, the number of local epochs, and the learning rate, respectively.

4. Related work

Since our work is the first to apply FL to predict events, we have a brief review on the FL with GPs and other typical methods in point process methods. In [30], GPs for few shot classification learn covariance functions parameterized by deep neural networks. In [31], GPs were applied to meta-learning by maximizing the mutual information between the query set and a latent representation of the support set. In [32], the GP FL integrates deep kernel learning and scalable random features. The hyperparameters of a global deep kernel is optimized while training local GPs [33]. DenseGP [34] uses inducing locations in GP for meta learning, while using the same kernel parameters for all the clients/domains. pFedBayes [35] may be closest to FedPP, which proposes a general framework of using Bayesian variational inference for FL. In addition to the application difference, pFedBayes uses an averaging strategy for global aggregation, which might be sub-optimal for the objective functions.

The TPPs with a nonlinear intensity function is important as it can model complex real-world patterns. These nonlinear functions include rectifier [36], exponential [37] and sigmoid [5,6]. The sigmoid mapping function has the advantage that the Pólya-Gamma augmentation scheme can be utilized to convert the likelihood into a Gaussian form, which makes the inference tractable. Also, there is a rich line of research on developing neural embedding methods [1,11,20,21,38,39] to study event data. Their main idea is to use various neural network architectures, including RNN, LSTM, and Transformer, to obtain historical embeddings, which differs substantially from our focus in this work.

Comparison with Neural Processes. Neural processes (NPs) and TPPs are both methods for modeling uncertainty, but they operate in different contexts. NPs [40,41] are a class of deep learning models that aim to learn distributions over functions. The "neural" in NPs refers to the use of neural networks to approximate these function distributions, making them suitable for tasks such as regression and classification with a focus on uncertainty quantification. While NPs have shown promise in a variety of applications, their use in federated learning with decentralized, heterogeneous data has not been extensively explored. On the other hand, TPPs are used to model the occurrence of events over time, commonly applied in sequential event-based tasks. Our approach leverages neural networks to capture event dependencies over time, enabling efficient uncertainty quantification in federated learning. Applying TPPs in the federated settings addresses the challenges of heterogeneous data and uncertainty in temporal event modeling.

Sequential Neural Processes (SNPs) [42] and TPPs target different objectives despite both being designed for sequence modeling. SNPs extends NPs to model the conditional distribution of sequential data, focusing on learning a mapping between inputs and outputs in a flexible and probabilistic manner. It assumes the input-output pairs are conditionally independent and is particularly suitable for capturing general sequence relationships without explicit modeling of temporal dependencies. In contrast, TPPs explicitly model the stochastic nature of event occurrences over time, emphasizing the temporal dynamics and inter-event dependencies. Our work leverages TPPs to model temporal event data, focusing on the dynamic relationships between event time and associated marks, which differs from SNPs of learning mappings for general sequence data.

The work in [43] also utilizes Poisson processes as a modeling tool, specifically applying recurrent Poisson processes to speech recognition tasks. While this approach shares similarities with our work, a key difference lies in the context and application. In contrast to this work, which focuses on sequential event occurrences in speech data, our work introduces the use of Poisson processes

within the framework of federated learning. Here, we model the uncertainty of decentralized clients' data using point processes, enabling us to capture complex temporal dependencies and to offer a scalable solution for uncertainty quantification in federated systems.

Comparison with Bayesian Deep Learning. Bayesian deep learning (BDL) provides a probabilistic framework for deep learning models, enabling them to capture uncertainty in both model parameters and prediction. BDL primarily focuses on integrating Bayesian methods, such as Bayesian neural networks (BNNs) and variational inference techniques, into deep learning models to address uncertainty quantification and improve model generalization [44]. Our work shares a foundational connection with BDL, as we also incorporate uncertainty quantification within our model using point processes. However, unlike traditional BDL approaches that typically rely on approximating the posterior distribution of model parameters (e.g., using BNNs or Monte Carlo methods), our method applies point processes in the context of federated learning. We specifically focus on modeling the uncertainty of decentralized, heterogeneous data across clients by capturing temporal dependencies and event-based dynamics. This allows for scalable and efficient uncertainty quantification, which is particularly challenging in federated settings where data is non-IID and distributed across a large number of clients.

5. Experiments

5.1. Data, models and settings

In the experiments, we utilize five benchmark datasets from diverse domains: *Taobao*, which contains timestamped user online shopping behaviors on the Taobao platform; *Retweet*, consisting of timestamped user retweet events; *Conttime*, a simulated public dataset provided by [38]; *Stack Overflow*, which includes question-answering badge events; and *Amazon*, featuring timestamped user purchase behaviors on the Amazon platform. These datasets are employed to evaluate the performance of our proposed model, FedPP, against baseline methods. For each sequence, we use the first 60 % for training, the next 20 % for validation (used for hyperparameter tuning), and the final 20 % for testing. Additionally, we normalize the entire observation timeline into $[0, 100]$ for numerical stability. To construct a heterogeneous setting, where local data across clients are non-IID, we allocate an equal number of samples to all clients, with each client being assigned $k (< K)$ different event types. More experimental details are provided in Appendix I.

Rationale for Dataset Selection. To comprehensively evaluate the performance and robustness of our proposed method, we selected five benchmark datasets: Taobao, Retweet, Conttime, Stack Overflow, and Amazon. These datasets are chosen for their relevance to federated learning challenges such as data heterogeneity, privacy, and sparsity. Taobao represents user behavior data in e-commerce scenarios, characterized by high heterogeneity and privacy sensitivity, making it ideal for personalized federated learning. Retweet captures sparse social network interactions, reflecting distribution imbalance commonly observed in federated systems. Conttime, a continuous time event dataset, emphasizes temporal dynamics and tests the model's ability to handle time-dependent patterns across non-IID client data. Stack Overflow highlights challenges in knowledge-sharing platforms, where user participation is highly imbalanced, while the Amazon dataset provides a large-scale benchmark with natural sparsity and privacy concerns, simulating real-world federated conditions. As a whole, these datasets enable us to validate the method's effectiveness under diverse, real-world scenarios.

Comparison Methods. FedPP is compared to the following baseline methods: (1) *RMTPP* [11], which encodes past events using an RNN and employs a mixture of *Gompertz* distributions for the probabilistic decoder; (2) *SAHP* [21], which models the intensity function with a self-attention mechanism and uses the *softplus* nonlinear transformation to ensure non-negativity of the intensity function; (3) *THP* [20], a self-attention-based point process model similar to SAHP; (4) *TCVAE* [39], a generative neural embedding method that utilizes a variational autoencoder (VAE) for probabilistic decoding; (5) *TCDDM* [39], which is similar to TCVAE but uses a diffusion model for probabilistic decoding; (6) *Dec-ODE* [45], which employs neural ordinary differential equations (ODE) to capture complex dynamics in the intensity function; (7) *LSTM-FedPP*, the FedPP model with an LSTM-based historical encoder; (8) *ATT-FedPP*, the FedPP model with an attention-based historical encoder. As stated earlier, FedPP with deep kernel learning is the primary model under evaluation, while LSTM-FedPP and ATT-FedPP are left for future work.

To facilitate fair and universally applicable comparisons, we implement federated modeling by applying parameter averaging to all the aforementioned original TPPs baseline methods, and prefixing them with "Fed" to indicate their federated versions.

Implementation. In all experiments, we set the total number of clients $C = 20$. For each communication round, we fix the number of (randomly selected) participating clients to be $S = 10$. All datasets are configured with a fixed number of 100 global communication rounds, which is based on the observation that further increasing the number of communication rounds does not result in a significant performance improvement. We employ Adam [46] as a basic optimizer and set the number of local epochs $E = 5$ for all methods.

Predictive Performance. We adopt the expected posterior predictive log-likelihood on the test time interval to compare all the models. In order to sufficiently validate the modeling advantages of FedPP, we make different settings to evaluate its predictive performance. Our code for reproducing the experiments is available at <https://github.com/HuiChen3313/FedPP>.

5.2. Performance comparison

Test Log-likelihood for Different Datasets. We conduct the experiments under heterogeneous settings on five different benchmark datasets. Note that, since there are no closed-form likelihoods for Fed-TCVAE and Fed-TCDDM, we report the variational lower

Table 1

Performance evaluation for test log-likelihood (mean $\pm$ std) on different benchmark datasets (**Bold** represents the top-3 performance).

Dataset	Taobao	Retweet	Conttime	Stack Overflow	Amazon
Fed-RMTPP	1.34 $\pm$ 0.12	-1.73 $\pm$ 0.06	1.57 $\pm$ 0.08	-1.37 $\pm$ 0.02	1.24 $\pm$ 0.13
Fed-SAHP	2.79 $\pm$ 0.02	-1.14 $\pm$ 0.12	2.45 $\pm$ 0.03	2.63 $\pm$ 0.08	3.92 $\pm$ 0.04
Fed-THP	2.75 $\pm$ 0.07	-1.93 $\pm$ 0.02	2.34 $\pm$ 0.07	2.61 $\pm$ 0.09	3.96 $\pm$ 0.02
Fed-TCVAE	$\geq 1.26 \pm 0.06$	$\geq -3.14 \pm 0.23$	$\geq 1.28 \pm 0.09$	$\geq 1.36 \pm 0.12$	$\geq 1.97 \pm 0.11$
Fed-TCDDM	$\geq 1.22 \pm 0.12$	$\geq -2.13 \pm 0.18$	$\geq 1.34 \pm 0.08$	$\geq 1.42 \pm 0.07$	$\geq 2.38 \pm 0.06$
Fed-Dec-ODE	2.88 $\pm$ 0.01	-1.35 $\pm$ 0.08	2.53 $\pm$ 0.04	2.63 $\pm$ 0.02	3.93 $\pm$ 0.03
FedPP	2.92 $\pm$ 0.04	-1.32 $\pm$ 0.16	2.42 $\pm$ 0.16	2.66 $\pm$ 0.01	4.07 $\pm$ 0.01
LSTM-FedPP	3.14 $\pm$ 0.05	-1.27 $\pm$ 0.13	2.44 $\pm$ 0.12	2.68 $\pm$ 0.07	4.12 $\pm$ 0.04
ATT-FedPP	3.07 $\pm$ 0.02	-1.23 $\pm$ 0.09	2.46 $\pm$ 0.08	2.71 $\pm$ 0.07	4.09 $\pm$ 0.03

Table 2

Ablation study for FedPP. KL and DK represent the aggregation by the KL divergence and the deep kernel, respectively. ‘None’ implies that neither of two strategies has been used, while ‘Both’ indicates that both strategies have been employed.

Dataset	Taobao	Retweet	Conttime	Stack Overflow	Amazon
None	2.75 $\pm$ 0.02	-1.40 $\pm$ 0.06	2.12 $\pm$ 0.08	2.54 $\pm$ 0.02	3.82 $\pm$ 0.04
w/KL	2.78 $\pm$ 0.02	-1.42 $\pm$ 0.05	2.15 $\pm$ 0.12	2.57 $\pm$ 0.04	3.84 $\pm$ 0.01
w/DK	2.82 $\pm$ 0.05	-1.38 $\pm$ 0.03	2.37 $\pm$ 0.03	2.59 $\pm$ 0.06	3.91 $\pm$ 0.02
Both	2.92 $\pm$ 0.04	-1.32 $\pm$ 0.16	2.42 $\pm$ 0.16	2.66 $\pm$ 0.01	4.07 $\pm$ 0.01

bounds¹ for them. The main results in Table 1 indicate that our proposed method FedPP and its variants LSTM-FedPP and ATT-FedPP dominate the other baselines on all datasets except Conttime, which demonstrates the feasibility and effectiveness of our local model framework and the global aggregation scheme, especially for real-world datasets. Additionally, we observe that incorporating neural historical encoders consistently improves the model performance across all datasets. The performance of different neural historical encoders is similar, which aligns with the conclusion in [2].

Ablation Study. FedPP takes two critical strategies to improve its performance: 1) For the server, we design the global aggregation guided by divergence such as the KL divergence, rather than the simple federated averaging, for a more reasonable global model, especially for heterogeneous data scenarios. 2) For each client, we add the deep kernel to transform inputs into the points in the input space of GP for a scalable and powerful uncertainty representation. To investigate the individual efficacy of these two strategies, we perform an ablation study on all datasets and report the test log-likelihood in Table 2. This result indicates that both strategies individually improve the model performance, and using both methods simultaneously yields the best results. This is because using a deep kernel helps model the input, while utilizing the KL aggregation assists in capturing the relationship between local models and the global model from a distributional perspective.

Effects of Different Global Aggregation Schemes. In addition to the KL divergence aggregation, the Wasserstein distance is also an alternative aggregation method. In this experiment, we investigate the impact of FedAvg, KL divergence aggregation, and Wasserstein distance aggregation on model performance across five different benchmark datasets. The results in Table 3 indicate that the improvement of model performance by the KL and Wasserstein distance aggregation is superior to FedAvg, and the KL aggregation scheme slightly outperforms the Wasserstein distance scheme on all datasets.

Main Results on Event Prediction. In our experiments, we primarily utilize the test log-likelihood as a key metric to evaluate model performance. Additionally, in the context of the point process, the ability to predict events is of particular importance and serves as another critical aspect of performance evaluation. Therefore, in this part, we introduce two additional evaluation metrics: root mean square error (RMSE) to evaluate the reliability of the predicted time points of events, and accuracy (ACC) to assess the correctness in predicting the event type. Specifically, we use the expected value $\mathbb{E}[r]$ as the predicted next time point and select the event type with the highest probability at time t as the predicted next event type. Table 4 presents the comparison results on three different benchmark datasets, from which we can see that LSTM-FedPP and ATT-FedPP consistently achieve the best performance across nearly all datasets, demonstrating the strong predictive capability of our method in federated environments. Predicting both occurrence timestamps and types of client events requires further strategies such as on their tradeoff and multiobjective designs.

Further Exploration on Divergence Metrics. In this paper, the KL divergence is primarily used as our global aggregation method due to its superior performance and computational simplicity. As a result, the other two divergence metrics, the Wasserstein distance and the maximum mean discrepancy (MMD), are not fully explored, particularly in the context of large-scale federated systems. To address this, we conduct additional experiments to evaluate the performance of these different divergence metrics in large-scale federated systems by varying the number of clients C from {100, 150, 200}. The main results are presented in Table 5, which show that the model performance decreases as the number of clients increases for all divergence metrics. However, the KL divergence consistently achieves the best performance across all datasets, followed by the Wasserstein distance, while the MMD shows the poorest

¹ The performance of the closed-form likelihood is superior to that of the reported variational lower bound.

Table 3

Comparison of test log-likelihood with different global aggregations on benchmark datasets.

Dataset	Taobao	Retweet	Conttime	Stack Overflow	Amazon
FedAvg	2.82 ± 0.03	-1.38 ± 0.05	2.37 ± 0.09	2.59 ± 0.03	3.91 ± 0.07
Wasserstein	2.91 ± 0.08	-1.35 ± 0.06	2.40 ± 0.12	2.61 ± 0.09	4.05 ± 0.07
KL	2.92 ± 0.04	-1.32 ± 0.16	2.42 ± 0.16	2.66 ± 0.01	4.07 ± 0.01

Table 4

The comparison of RMSE and ACC on Taobao, Stack Overflow and Amazon datasets. (**Bold** represents the best result).

Dataset	Taobao		Stack Overflow		Amazon	
Metric	RMSE	ACC	RMSE	ACC	RMSE	ACC
Fed-RMTPP	0.47 ± 0.16	60.13 ± 0.15	1.57 ± 0.04	58.42 ± 0.02	0.63 ± 0.12	51.37 ± 0.07
Fed-SAHP	0.49 ± 0.07	58.24 ± 0.08	1.63 ± 0.06	56.17 ± 0.12	0.65 ± 0.06	52.44 ± 0.02
Fed-THP	0.52 ± 0.03	60.98 ± 0.05	1.61 ± 0.09	60.32 ± 0.04	0.61 ± 0.07	51.05 ± 0.11
Fed-TCVAE	0.32 ± 0.08	61.55 ± 0.08	1.55 ± 0.03	60.18 ± 0.12	0.62 ± 0.04	49.22 ± 0.08
Fed-TCDDM	0.45 ± 0.04	62.44 ± 0.07	1.57 ± 0.03	57.24 ± 0.02	0.57 ± 0.05	51.48 ± 0.13
Fed-Dec-ODE	0.36 ± 0.09	59.13 ± 0.11	1.55 ± 0.03	61.26 ± 0.07	0.60 ± 0.03	54.55 ± 0.08
LSTM-FedPP	0.27 ± 0.07	63.78 ± 0.05	1.56 ± 0.04	61.45 ± 0.14	0.61 ± 0.03	56.14 ± 0.03
ATT-FedPP	0.35 ± 0.02	64.27 ± 0.04	1.51 ± 0.17	57.41 ± 0.03	0.63 ± 0.02	56.31 ± 0.07

Table 5

The comparison of test log-likelihood for different divergence metrics on Taobao, Stack Overflow and Amazon datasets. (**Bold** represents the best result).

Dataset	Taobao			Stack Overflow			Amazon		
Number of Clients	$C = 100$	$C = 150$	$C = 200$	$C = 100$	$C = 150$	$C = 200$	$C = 100$	$C = 150$	$C = 200$
Wasserstein	0.54 ± 0.08	0.51 ± 0.06	0.49 ± 0.11	0.18 ± 0.09	0.16 ± 0.08	0.12 ± 0.09	1.39 ± 0.05	1.35 ± 0.06	1.33 ± 0.08
MMD	0.37 ± 0.13	0.34 ± 0.12	0.31 ± 0.16	0.12 ± 0.13	0.09 ± 0.14	0.07 ± 0.11	0.98 ± 0.14	0.94 ± 0.13	0.88 ± 0.16
KL	0.56 ± 0.04	0.54 ± 0.02	0.52 ± 0.01	0.22 ± 0.01	0.19 ± 0.02	0.18 ± 0.01	1.42 ± 0.03	1.37 ± 0.02	1.35 ± 0.01

performance with a relatively high standard deviation. This indicates that the performance of MMD is less stable. This outcome may be attributed to the fact the computational process of MMD is more complex compared to that of the other two divergence metrics, as the MMD requires obtaining the true optimal solution for μ and σ through optimization algorithms, which is challenging to achieve in practice.

5.3. Further evaluation on federated settings

Since this work primarily explores the feasibility and effectiveness of the TPPs in federated scenarios, it is crucial to investigate the performance of the proposed method FedPP in various federated settings.

5.3.1. Main results on local epochs

In general, there is a trade-off between local training steps and global communication rounds in FL, which means that adding more computation to each client in each communication round may reduce the overall communication cost of the FL system. In this experiment, we explore this characteristic by varying the values of the local epochs E . Specifically, we conduct the experiment on several benchmark datasets and set the number of local epochs between communication rounds over $\{1, 2, 5, 10\}$ with the total communication rounds fixed. The results are shown in Fig. 2 (left). Except for Stack Overflow, more local epochs lead to the worse performance. Particularly for Conttime, more local epochs result in an obvious performance degradation. However, it is worth noting that a balance between computation and communication costs is essential in selecting local epochs.

5.3.2. Main results on highly data-scarce cases

To evaluate the impact of local data size on the model performance, we conduct the experiment on highly data-scarce cases. Based on the default experimental data size, we further vary the proportion of local data size for each client over $\{1\%, 5\%, 10\%, 20\%\}$ on Taobao, Stack Overflow, and Amazon. The results of different datasets are reported in Fig. 2 (middle). It can be found that the absence of data does make a certain impact on the model performance, especially in the case of the 1% local data size, where the model performance experiences a severe degradation. However, when the data size is increased by just 5%, our method obtains relatively good results, demonstrating the superior performance of our method in such extreme federated settings.

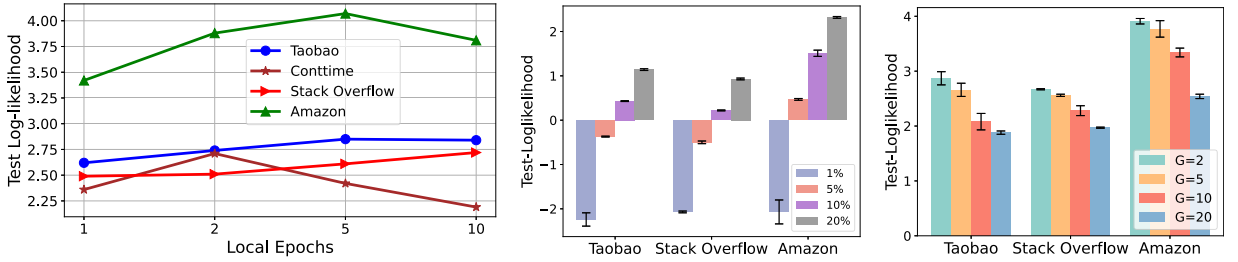


Fig. 2. (Left) Performance comparison of test log-likelihood on four different datasets with varying local epochs. (Middle) Performance comparison of test log-likelihood on three different datasets by varying the local data size. (Right) Performance comparison w.r.t. test log-likelihood on three different datasets by varying the struggling period.

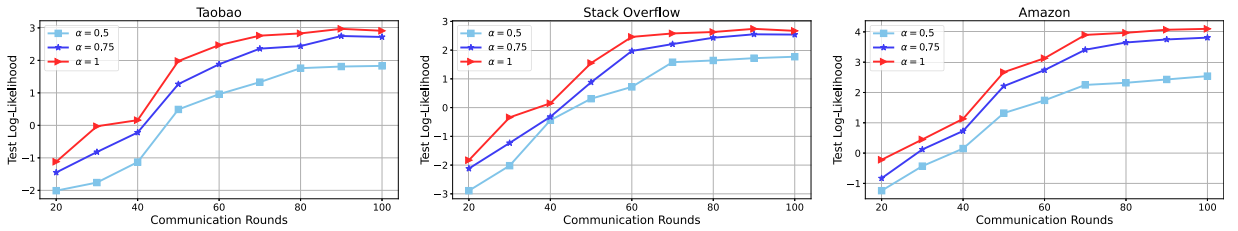


Fig. 3. Performance comparison of test log-likelihood on three real-world datasets with different levels of data heterogeneity.

5.3.3. Main results on straggling scenarios

As we know, FL is a dynamic learning system where client participation varies randomly across communication rounds. A highly challenging case arises when some clients undergo prolonged straggling-remaining inactive for many consecutive rounds-thus causing a long-term absence of their local training samples [47]. To explore the robustness of the proposed approach in such scenarios, we conduct the experiments on Taobao, Stack Overflow and Amazon and set the straggling period G to 2, 5, 10 and 20 communication rounds respectively for comparison. Namely, the activated clients and inactivated clients remain fixed for every G communication rounds. Fig. 2 (right) presents the test log-likelihood on three different real-world datasets, from which we can see that larger straggling periods G has a stronger negative impact on the model performance. This is mainly because some inactive clients have important features that do not present in currently participating clients, causing bias in the global model learned during the straggling period. Additionally, we observe that our method achieves relatively smooth results on these three datasets, especially on Stack Overflow. This demonstrates the stability and reliability of our method in such FL scenarios.

5.3.4. Main results on heterogeneous data

To quantify the degree of non-IIDness [24], we incorporate established metrics by leveraging the Dirichlet distribution to simulate varying levels of data heterogeneity among clients. This approach provides a more comprehensive evaluation of the robustness of our method on heterogeneous data. Specifically, we set the class probabilities in each client by sampling from a Dirichlet distribution with the same concentration parameter α . By varying $\alpha \in \{0.5, 0.75, 1\}$ across experiments, where smaller α values correspond to higher data heterogeneity, we simulate different degrees of non-IIDness among clients. Fig. 3 shows the test log-likelihood on three real-world datasets, from which we can observe that the model performance decreases as data heterogeneity increases. However, this decrease remains within a reasonable range, demonstrating that our method exhibits strong robustness across different levels of data heterogeneity.

5.3.5. Main results on system heterogeneity

In a practical federated learning setup, where a server collaborates with numerous clients like smartphones, tablets, and laptops. The increase of the number of clients enhances the heterogeneity of the system, resulting in a degradation of model performance. To this end, we evaluate the scalability of our approach on three real-world datasets by adjusting the number of clients C over $\{50, 100, 150, 200\}$. To adhere to a cross-device configuration, we set the number of clients participating in each communication round to 10% of the total number of clients. The convergence results are shown in Fig. 4, from which we can see that as the number of clients increases, the performance of the model across three datasets is deteriorating. This is mainly because, with a constant number of samples, the increase of the number of clients leads to stronger data heterogeneity among different clients, which in turn enlarges the discrepancy between the aggregated global model and the optimal global model. However, it is noted that the degradation in model performance with FedPP is not significant in this scenario, which demonstrates the robustness and practicality of our approach.

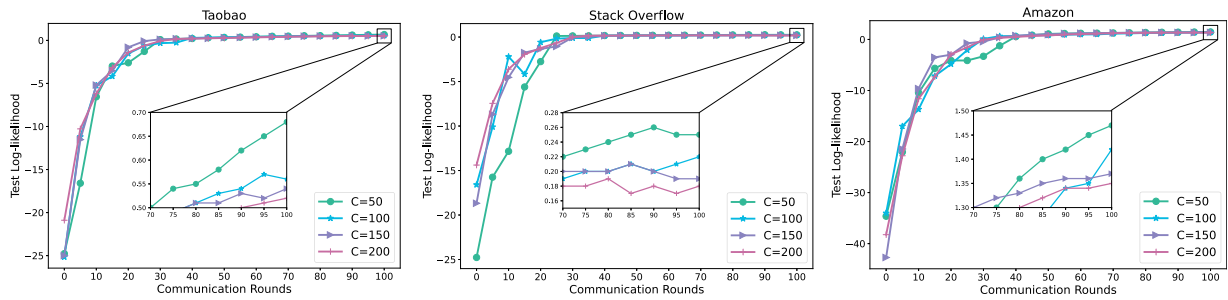


Fig. 4. Test log-likelihood over communication rounds on Taobao, Stack Overflow and Amazon datasets with varying number of clients.

6. Conclusions

We have introduced FedPP, the first approach to learning point processes for uncertain and sparse events in a federated and privacy-preserving manner. Extensive experimental evaluations show that FedPP consistently outperforms federated versions of benchmark methods across five datasets. The integration of neural embeddings into sigmoidal Gaussian Cox Processes proves highly effective in both summarizing historical information and learning flexible intensity functions. Additionally, we have proposed a novel framework for global aggregation using divergence measures, offering practical interpretation of the aggregation process. This divergence-guided framework can be extended to any Bayesian federated learning method that involves probability distribution aggregation, paving the way for future work in this direction. Finally, our current algorithm avoids sharing raw data but may remain vulnerable to advanced privacy attacks. Further studies on jointly modeling different aspects of event factors and incorporating privacy-preserving strategies into FedPP-like models would foster some of future directions.

CRedit authorship contribution statement

Hui Chen: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Conceptualization; **Xuhui Fan:** Methodology, Investigation, Formal analysis, Conceptualization; **Hengyu Liu:** Validation, Conceptualization; **Yaqiong Li:** Validation, Conceptualization; **Zhilin Zhao:** Writing – review & editing; **Feng Zhou:** Writing – review & editing; **Christopher John Quinn:** Writing – review & editing; **Longbing Cao:** Writing – review & editing, Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research has been sponsored in part by the Australian Research Council Discovery grant DP240102050, ARC LIEF grant LE240100131, and ARC Linkage grant LP230201022.

Appendix A. Related work on survival analysis

Survival analysis is a statistical framework used to analyze the time until an event of interest occurs, commonly employed in fields such as healthcare, finance, and engineering to model time-to-event data [48,49]. Traditional survival analysis often relies on techniques like the Cox proportional hazards model, where the risk (or hazard) of an event is estimated over time. In this context, events are usually assumed to occur only once, and the data might involve censoring, where the event does not occur during the observation period. When extended to FL, survival analysis must adapt to decentralized settings where data is distributed across multiple clients, such as hospitals or devices [50], without sharing raw data for privacy reasons [51,52]. In federated learning-based survival analysis, each client locally trains a survival model on its data, and the central server aggregates the learned models without accessing individual-level data, thus maintaining privacy.

In contrast, TPPs focus on predicting multiple recurring events over time, such as transaction patterns, server requests, or patient visits. Unlike survival analysis, which typically handles a single event per subject and often deals with censoring, TPPs model event intensity functions to capture sequential occurrences and inter-event dependencies. When extended to federated learning, TPP-based

models face similar privacy-preserving challenges, but with the added complexity of modeling dynamic temporal relationships between multiple events. This distinction makes TPP-based federated models more flexible in applications where multiple events are expected, but it also demands more sophisticated techniques to capture the intricate dependencies and ensure global model aggregation across clients. Hence, while both survival analysis and TPP share common goals in event prediction, the nature of events and the modeling approaches differ significantly in their federated learning extensions.

Appendix B. Deep kernel learning

Modern GP approaches, such as Deep Gaussian Processes (DGPs) [53] and Deep Kernel Learning (DKL) [54], usually use deep architectures to enhance the expressive power of kernels. For instance, DKL defines kernels on features mapped by a neural network $g_{\tilde{\mathbf{w}}}(\cdot)$, with $\tilde{\mathbf{w}}$ being the neural network parameters. In this way, the RBF kernel entry between $\mathbf{x}_i$ and $\mathbf{x}_j$, as an example, can be formulated as $\kappa_{\mathbf{x}_i, \mathbf{x}_j; [\tilde{\mathbf{w}}; r; l]} = r \cdot \exp(-\|g_{\tilde{\mathbf{w}}}(\mathbf{x}_i) - g_{\tilde{\mathbf{w}}}(\mathbf{x}_j)\|^2 / (2l^2))$, with r and l being the scaling and band-width parameters. Without particularly specified, we use the RBF kernel as the default kernel in this work.

Appendix C. Additivity property of TPPs

Property Appendix C.1. Let $N_1(t)$ and $N_2(t)$ be two independent temporal point processes with intensity functions $\lambda_1(t)$ and $\lambda_2(t)$ respectively. The process $N(t) = N_1(t) + N_2(t)$ is also a temporal point process with the intensity function $\lambda(t) = \lambda_1(t) + \lambda_2(t)$.

Appendix D. Sparse Gaussian processes

Although powerful and highly flexible, GP, which is the key component in SGCPs, is often questioned due to its high computational cost. In particular, obtaining the posterior of the latent function requires inverting the kernel matrix $\kappa_{\mathbf{x}_1:N, \mathbf{x}_1:N; \mathbf{w}}$ of N training data points $\mathbf{x}_1:N$, resulting in a computational cost scaled to $\mathcal{O}(N^3)$. Introducing sparse inducing points $\mathbf{u}$ is a common approximation strategy to mitigate this issue. Specifically, the kernel matrix $\kappa_{\mathbf{z}, \mathbf{z}; \mathbf{w}}$ over M inducing inputs $\mathbf{z}_1:M$ can be regarded as a low-rank approximation to the full kernel matrix $\kappa_{\mathbf{x}_1:N, \mathbf{x}_1:N; \mathbf{w}}$, reducing the computational cost from $\mathcal{O}(N^3)$ to $\mathcal{O}(NM^2)$. Given inducing inputs $\mathbf{z}$ and inducing points $\mathbf{u}$, in which $\mathbf{u}$ is generated as $\mathbf{u} \sim \mathcal{N}(\nu(\mathbf{z}), \kappa_{\mathbf{z}, \mathbf{z}; \mathbf{w}})$, we can have the approximated random function value $\tilde{f}(\cdot)$ following the Gaussian distribution as:

$$\tilde{f}(\cdot) | \mathbf{w}, \mathbf{u}, \mathbf{z} \sim \mathcal{N}(\tilde{f}(\cdot); \nu(\cdot) + \kappa_{\cdot, \mathbf{z}; \mathbf{w}} \kappa_{\mathbf{z}, \mathbf{z}; \mathbf{w}}^{-1} (\mathbf{u} - \nu(\mathbf{z})), \kappa_{\cdot, \cdot; \mathbf{w}} - \kappa_{\cdot, \mathbf{z}; \mathbf{w}} \kappa_{\mathbf{z}, \mathbf{z}; \mathbf{w}}^{-1} \kappa_{\mathbf{z}, \cdot; \mathbf{w}}). \quad (\text{D.1})$$

Appendix E. Steps of the inference

Step 1, Calculating $\mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)}[\log p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c)]$. Since we model the intensity function of client c as $\lambda_c(t) = m_c \cdot \sigma(f_c(t))$, the likelihood of the event $\mathbf{t}_c$ can be approximated as:

$$p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c) \stackrel{(i)}{\approx} \int p(\mathbf{t}_c | \tilde{f}_c(\cdot), m_c) p(\tilde{f}_c(\cdot) | \mathbf{u}_c, \mathbf{w}_c, \mathbf{z}) p(\mathbf{u}_c | \mathbf{w}_c, \mathbf{z}) d\mathbf{u}_c d\tilde{f}_c(\cdot) \quad (\text{E.1})$$

in which $p(\mathbf{t}_c | \tilde{f}_c(\cdot), m_c)$ can be written as:

$$p(\mathbf{t}_c | \tilde{f}_c(\cdot), m_c) = \exp\left(-\int_0^T m_c \sigma(f_c(t)) dt\right) \prod_{i=1}^{n_c} m_c \sigma(f_c(t_{c,i})). \quad (\text{E.2})$$

In Eq. (E.1), we first augment the original likelihood $p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c)$ with random function $\tilde{f}_c(\cdot)$ and inducing points $\mathbf{u}_c$. The approximation (i) in Eq. E.1 denotes that the distribution of random function $p(f_c)$ is approximated by $\tilde{P}(f_c)$, which is supported by the sparse inducing points $\mathbf{u}_c$ (see details in Eq. (D.1)).

Eq. (E.2) contains integration and sigmoidal operations in the exponential part, which makes it difficult to directly obtain a closed-form representation for inference schedules. Adapting the SGCPs methods [17–19,26] and further augmenting latent marked Poisson processes Π_c , which has $\nu \times P_{\text{PG}}(\xi | 1, 0)$ as the intensity function in $[0, T] \times \mathbb{R}^+$, and Polya-Gamma random variables $\{\xi_{c,i}\}_{i=1}^{n_c}$, which has the prior distribution $P_{\text{PG}}(\xi | 1, 0)$, into Eq. (E.1), we have

$$\begin{aligned} p(\mathbf{t}_c | \tilde{f}_c(\cdot), m_c) &= \int p(\mathbf{t}_c, \Pi_c, \{\xi_{c,i}\}_{i=1}^{n_c} | m_c, \tilde{f}_c(\cdot)) d\Pi_c d\{\xi_{c,i}\}_{i=1}^{n_c} \\ &= \int e^{-m_c T} \cdot \prod_{i=1}^{n_c} \left[m_c e^{g(\xi_{c,i}, \tilde{f}_c(t_{c,i}))} P_{\text{PG}}(\xi_{c,i} | 1, 0) \right] \prod_{(t_{c,j}, \xi_{c,j}) \sim \Pi_c} \left[e^{g(\xi_{c,j}, -\tilde{f}_c(t_{c,j}))} m_c P_{\text{PG}}(\xi_{c,j} | 1, 0) \right] \cdot d\Pi_c d\{\xi_{c,i}\}_{i=1}^{n_c} \end{aligned} \quad (\text{E.3})$$

in which $g(\xi, f) = f/2 - \xi f^2/2 - \log 2$.

In short, through augmenting the random function $\tilde{f}_c(\cdot)$, inducing points $\mathbf{u}_c$, latent marked Poisson processes Π_c , and Polya-Gamma random variables $\{\xi_{c,i}\}_{i=1}^{n_c}$, we are able to obtain a closed-form representation of ELBO for client c , which is:

$$\mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)}[\log p(\mathbf{t}_c | \mathbf{w}_c, \mathbf{z}, m_c)] - \text{DV} \left[q_{\Phi_c}(\mathbf{w}_c) \parallel p(\mathbf{w}_c) \right]$$

$$\begin{aligned}
& \geq \mathbb{E}_{q(\tilde{f}_c(\cdot))} [\log P(\mathbf{t}_c | m_c, \tilde{f}_c(\cdot))] \\
& \quad - \text{KL}[q(\mathbf{u}_c, \tilde{f}_c(\cdot)) \parallel P(\mathbf{u}_c, \tilde{f}_c(\cdot))] - \text{DV}[q_{\Phi_c}(\mathbf{w}_c) \parallel p(\mathbf{w}_c)] \\
& \geq \mathbb{E}_{q(\cdot)} [\log P(\mathbf{t}_c | \Pi_c, \{\xi_{c,i}\}_{i=1}^{n_c}, m_c, \tilde{f}_c(\cdot))] - \text{KL}[q(\mathbf{u}_c, \tilde{f}_c(\cdot)) \parallel P(\mathbf{u}_c, \tilde{f}_c(\cdot))] \\
& \quad - \text{KL}[q(\Pi_c, \{\xi_{c,i}\}_{i=1}^{n_c}) \parallel P(\Pi_c, \{\xi_{c,i}\}_{i=1}^{n_c})] - \text{DV}[q_{\Phi_c}(\mathbf{w}_c) \parallel p(\mathbf{w}_c)] \\
& = \text{ELBO}_c.
\end{aligned} \tag{E.4}$$

In fact, we are able to obtain closed-form representations for all the terms involved in Eq. (E.4) and then use mean-field variational inference to obtain closed-form updating equations for the variational distributions of these random variables.

Step 3, Maximizing the Objective Function of Client c with Respect to $q_{\Phi_c}(\mathbf{w}_c)$. We use the first-order stochastic gradient descent (SGD) algorithms to optimize the ELBO with respect to $q_{\Phi_c}(\mathbf{w}_c)$. For problem (5) of clients, we can get the closed-form results for KL divergence terms but cannot get that for the other term. To speedup the convergence, we use the minibatch gradient descent (GD) algorithm. So the stochastic estimator for the c th client is given. Regarding $q_{\Phi_c}(\mathbf{w}_c)$, its corresponding terms in ELBO_c can be abstracted as:

$$\begin{aligned}
& - \mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)} [\text{DV}[q(\mathbf{u}_c) \parallel p(\mathbf{u}_c | \mathbf{w}_c, \mathbf{z})]] \\
& + \mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)} [\log p(f_c(\cdot) | \mathbf{w}_c, \mathbf{z})] - \text{DV}[q_{\Phi_c}(\mathbf{w}_c) \parallel p(\mathbf{w}_c)].
\end{aligned} \tag{E.5}$$

We set the variational distribution $q_{\Phi_c}(\mathbf{w}_c)$ as

$$q_{\Phi_c}(\mathbf{w}_c) = \mathcal{N}([\mathbf{w}; \mathbf{z}]; [\mathbf{r}_\theta; \mathbf{r}_z], \text{diag}([\delta_\theta^2; \delta_z^2])). \tag{E.6}$$

The KL divergence term can be obtained in an analytical format since both $q_c = (\mathbf{w}, \mathbf{z})$ and $\pi_c = (\mathbf{w}, \mathbf{z})$ are Gaussian distributions, which is:

$$\text{DV}[q_{\Phi_c}(\mathbf{w}_c) \parallel p(\mathbf{w}_c)] \propto \frac{1}{2} \sum_m \left(\frac{(\boldsymbol{\mu}_{\theta,z,m} - \mathbf{r}_{\theta,z,m})^2}{\boldsymbol{\sigma}_m^2} + \frac{\delta_m^2}{\sigma_m^2} - \log \frac{\delta_m^2}{\sigma_m^2} \right).$$

Regarding the first term in Eq. (E.5), we can use reparameterization trick to formulate it as:

$$\begin{aligned}
& \mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)} [\text{DV}[q(\mathbf{u}_c) \parallel p(\mathbf{u}_c | \mathbf{w}_c, \mathbf{z})]] \\
& = \frac{1}{2} \mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)} \left[\log \frac{|\kappa_{\mathbf{z},\mathbf{z},\mathbf{w}}|}{|\Sigma_s|} + \text{trace}(\kappa_{\mathbf{z},\mathbf{z},\mathbf{w}}^{-1} \Sigma_s) + (\boldsymbol{\mu}_s - \boldsymbol{\nu})^\top \kappa_{\mathbf{z},\mathbf{z},\mathbf{w}}^{-1} (\boldsymbol{\mu}_s - \boldsymbol{\nu}) \right] \\
& = \frac{1}{2} \mathbb{E}_{\epsilon_\theta, \epsilon_z \sim \mathcal{N}(0,1)} \left[\log \frac{|\kappa_{\mathbf{z},\mathbf{z},\mathbf{w}}|}{|\Sigma_s|} + \text{trace}(\kappa_{\mathbf{z},\mathbf{z},\mathbf{w}}^{-1} \Sigma_s) + (\boldsymbol{\mu}_s - \boldsymbol{\nu})^\top \kappa_{\mathbf{z},\mathbf{z},\mathbf{w}}^{-1} (\boldsymbol{\mu}_s - \boldsymbol{\nu}) \right]
\end{aligned} \tag{E.7}$$

Regarding the second term in Eq. (E.5), it can be formulated as:

$$\mathbb{E}_{q_{\Phi_c}(\mathbf{w}_c)} [\log p(f_c(\cdot) | \mathbf{w}_c, \mathbf{z})] \propto \int \mathbb{E}_q \left[-\frac{f(t)}{2} + \frac{f^2(t)}{2} \xi \right] \Lambda(t, \xi) dt + \sum_n \mathbb{E}_q \left[\frac{f(t_n)}{2} + \frac{f^2(t_n)}{2} \xi_n \right]. \tag{E.8}$$

Appendix F. Details of mean-field variational inference

Optimal Polya-Gamma Density $q(\xi_{c,m})$.

$$\log q(\xi_{c,m}) = \mathbb{E}_q [-[f_c]^2(t_m) \xi_{c,m}/2] + \log P_{\text{PG}}(\xi_{c,m} | 1, 0) + \text{const} \tag{F.1}$$

Thus, we get

$$q(\xi_{c,m}) \propto \exp [-\mathbb{E}_q ([f_c]^2(t_m)) \xi_{c,m}/2] \cdot P_{\text{PG}}(\xi_{c,m} | 1, 0) \tag{F.2}$$

which leads to

$$q(\xi_{c,m}) = P_{\text{PG}} \left(\xi_{c,m} | 1, \sqrt{\mathbb{E}_q ([f_c]^2(t_m))} \right) \tag{F.3}$$

We can let $c_{c,m} = \sqrt{\mathbb{E}_q ([f_c]^2(t_m))}$.

Optimal Poisson Process $q(\Pi_c)$. Using the mean-field updating mechanism, we get the rate functions for the latent marked Poisson processes as:

$$\Lambda_c(t, \xi) = \frac{\exp(\mathbb{E}_q [\log v_c] - \frac{\mathbb{E}_q [f_c(t)]}{2})}{2 \cosh(\frac{c_c(t)}{2})} P_{\text{PG}}(\xi | 1, c_c(t)) \tag{F.4}$$

where $c_c(t) = \sqrt{\mathbb{E}_q [f_c(t)]^2}$. Again, we emphasize that the support of $\Lambda_c(t, \xi)$ is $(0, T] \times \mathbb{R}^+$.

Optimal Gaussian Processes $f_c(t)$. For the notation convenience, we neglect script c in the following discussion.

$$\log q(f) \propto \exp(U(f)) \quad (\text{F.5})$$

$U(f)$ is defined as:

$$U(f) = \mathbb{E}_q \left[\sum_{(x,\xi)_n \in \Pi} h(\xi_n, -f(x_n)) \right] + \sum_m \mathbb{E}_q [h(\xi_m, f(t_m))] \quad (\text{F.6})$$

$$= -\frac{1}{2} \int_{\mathcal{T}} A(t) f(t) dt + \int_{\mathcal{T}} B(x) f^2(t) dt \quad (\text{F.7})$$

where

$$A(t) = \sum_m \mathbb{E}_q [\xi_m] \delta(t - t_m) + \int_0^\infty \xi \Lambda(t, \xi) d\xi \quad (\text{F.8})$$

$$B(t) = \frac{1}{2} \sum_m \delta(t - t_m) - \frac{1}{2} \int_0^\infty \Lambda(t, \xi) d\xi \quad (\text{F.9})$$

in which we have the following equations for the above integration:

$$\mathbb{E}_q [\xi_m] = \frac{1}{2c_m} \tanh \left(\frac{c_m}{2} \right) \quad (\text{F.10})$$

$$\int_{\mathcal{T}} \Lambda(t, \xi) d\xi = \Lambda(t) \quad (\text{F.11})$$

$$\int_{\mathcal{T}} \xi \Lambda(t, \xi) d\xi = \frac{1}{2c(t)} \tanh \left(\frac{c(t)}{2} \right) \Lambda(t) \quad (\text{F.12})$$

By using sparse Gaussian process methods and its extension to infinite dimensional problems [55,56], we get the sparse posterior distribution for the function values at the inducing points as:

$$q(\mathbf{u}) = \mathcal{N}(\boldsymbol{\mu}_s, \Sigma_s) \quad (\text{F.13})$$

where the covariance matrix and mean are expressed as:

$$\Sigma_s = \left[\kappa_{\mathbf{z},\mathbf{z};\mathbf{w}}^{-1} \int_{\mathcal{T}} A(t) \kappa_{\mathbf{z},t;\mathbf{w}} \kappa_{t,\mathbf{z};\mathbf{w}} dt \kappa_{\mathbf{z},\mathbf{z};\mathbf{w}}^{-1} + \kappa_{\mathbf{z},\mathbf{z};\mathbf{w}}^{-1} \right]^{-1} \quad (\text{F.14})$$

$$\boldsymbol{\mu}_s = \Sigma_s \left(\kappa_{\mathbf{z},\mathbf{z};\mathbf{w}}^{-1} \int_{\mathcal{T}} \tilde{B}(t) \kappa_{\mathbf{z},t;\mathbf{w}} dt + \kappa_{\mathbf{z},\mathbf{z};\mathbf{w}}^{-1} \boldsymbol{\nu} \right) \quad (\text{F.15})$$

where $\tilde{B}(t) = B(t) - A(t)(\boldsymbol{\nu} - \kappa_{t,\mathbf{z};\mathbf{w}} \kappa_{\mathbf{z},\mathbf{z};\mathbf{w}}^{-1} \boldsymbol{\nu})$.

Appendix G. Divergence

The proofs of the following propositions follow immediately.

Proposition 1. Given $x_1 \sim \mathcal{N}(x_1; \mu_1, \sigma_1^2)$, $x_2 \sim \mathcal{N}(x_2; \mu_2, \sigma_2^2)$ and for any $\delta > 0$, we can obtain the following:

$$\mathbb{E}_{x_1 \sim \mathcal{N}(x_1; \mu_1, \sigma_1^2), x_2 \sim \mathcal{N}(x_2; \mu_2, \sigma_2^2)} \left[e^{-\frac{(x_1 - x_2)^2}{\delta^2}} \right] = \delta \cdot \frac{e^{-\frac{(\mu_1 - \mu_2)^2}{\delta^2 + 2\sigma_1^2 + 2\sigma_2^2}}}{\sqrt{\delta^2 + 2\sigma_1^2 + 2\sigma_2^2}} \quad (\text{G.1})$$

Proposition 2. Given $\mathcal{N}(\mu_1, \sigma_1^2)$, $\mathcal{N}(\mu_2, \sigma_2^2)$ and for any $\delta > 0$, the maximum mean discrepancy (MMD) of the RBF kernel can be expressed as:

$$\text{MMD}(\mathcal{N}(\mu_1, \sigma_1^2) \| \mathcal{N}(\mu_2, \sigma_2^2)) \quad (\text{G.2})$$

$$= \delta \cdot \frac{1}{\sqrt{\delta^2 + 4\sigma_1^2}} + \delta \cdot \frac{1}{\sqrt{\delta^2 + 4\sigma_2^2}} - 2\delta \cdot \frac{e^{-\frac{(\mu_1 - \mu_2)^2}{\delta^2 + 2\sigma_1^2 + 2\sigma_2^2}}}{\sqrt{\delta^2 + 2\sigma_1^2 + 2\sigma_2^2}} \quad (\text{G.3})$$

Proposition 3. $\sum_{c=1}^C \text{MMD}[q_{\Phi_c}(\mathbf{w}_c) \| p_{\Theta}(\mathbf{w}_c)]$ reaches its minimum for $\boldsymbol{\theta} = \{\boldsymbol{\mu}, \boldsymbol{\sigma}^2\}$ when $\boldsymbol{\theta}$ is valued at:

$$\boldsymbol{\mu}, \boldsymbol{\sigma}^2 := \arg \min_{\boldsymbol{\mu}, \boldsymbol{\sigma}^2} \sum_{c=1}^C \frac{1}{\sqrt{\delta^2 + 4\boldsymbol{\sigma}^2}} - 2 \frac{e^{-\frac{(\mathbf{r}_c - \boldsymbol{\mu})^2}{\delta^2 + 2\delta_c^2 + 2\boldsymbol{\sigma}^2}}}{\sqrt{\delta^2 + 2\delta_c^2 + 2\boldsymbol{\sigma}^2}} \quad (\text{G.4})$$

Algorithm 1 FedPP: Federated point process algorithm.

Input: J -communication rounds, E -local epochs, $\mathbb{S}$ -random subset of all clients. Initialize: μ^0 and σ^0 .

Server executes:

for $j = 0, 1, \dots, J - 1$ **do**

Sample $\mathbb{S}^j$ clients with size S uniformly at random

for each client $c \in \mathbb{S}^j$ **in parallel do**

$\phi_c^{j+1} \leftarrow \text{Client Update}(c, \theta^{(j)})$

end for

$\theta^{(j+1)} = \arg \min_{\theta} \sum_{c \in \mathbb{S}^j} \text{DV} \left[q_{\phi_c(\omega_c)} \parallel p_{\theta}(\omega_c) \right]$

end for

Client Update($c, \theta^{(j)}$):

for $e = 1, 2, \dots, E$ **do**

Update variational distributions of $\mathbf{u}_c$, ξ_c and Π_c using Eq. (21)

Update ϕ_c using Eq. (23)

end for

Return $\phi_c^{(j)} := \phi^{(E)}$

Appendix H. Algorithm

Algorithm 1 shows the training procedure of FedPP. defH.5A.5

Appendix I. Experimental details

Benchmark Datasets. We choose five benchmark datasets to evaluate our method: (1) *Taobao*, which is a public dataset generated for the 2018 Tianchi Big Data Competition. It comprises timestamped behavioral records (such as browsing and purchasing activities) of anonymized users on the online shopping platform Taobao. The data spans from November 25, to December 03, 2017. The dataset comprises a total of 5318 sequences and $K = 17$ types of events. (2) *Retweet*, which contains a total of 24,000 retweet sequences, where each sequence is composed of events represented as tuples, indicating tweet types and their corresponding times. There are $K = 3$ distinct types of retweeters: small, medium, and large. To classify retweeters into these categories, small retweeters have fewer than 120 followers, medium retweeters possess more than 120 but fewer than 1363 followers, and the remaining retweeters are labeled as large. The dataset provides information on when a post will be retweeted and by which type of user. (3) *Conttime*, which is a public dataset releases by [38]. There are 9000 sequences with a total of $K = 5$ event types. (4) *Stack Overflow*, which is a public dataset that encompasses sequences of user awards spanning a two-year period. In the Stack Overflow question-answering platform, recognizes users through awards based on their contributions, including posing insightful questions and providing valuable answers. The dataset comprises a total of 6633 sequences and $K = 22$ types of events. (5) *Amazon*, which is a public dataset similar to Taobao, containing a timestamped behavioral record of anonymized users on the online shopping platform Amazon. There are 14,759 sequences with a total of $K = 16$ event types.

Table I.1 displays the general characteristics of each dataset.

Training Details. We set the number of clients C to 20 and random subset of all clients S to 10 for all approaches. In addition, we perform 5 epochs to train local model for each client. In heterogeneous setting, the number of event types for each client is set to $k = 2$ for Retweet and Conttime, and $k = 4$ for other datasets. For the deep kernel function, we use 1-layer Bayesian neural network as the deep learning module. For the historical encoder, we set the layer number and embedding size to 2 and 32, respectively. We tune the learning rate η over $\{1e-3, 5e-3, 1e-2, 5e-2, 0.1\}$ and fix the rate to 1e-3. All experiments in this paper are implemented on a system comprising 4 cores, with each core powered by an Intel(R) Xeon(R) CPU E5-2686 at a frequency of 2.30GHz. The system is further enhanced by the inclusion of one NVIDIA Tesla K80 GPU. In addition to the GPU memory, the system boasts a total of 60GB of memory.

Table I.1
The statistics of five benchmark datasets.

Dataset	# of sequences	Types- K	Sequence Length		
			Min	Mean	Max
Taobao	4800	17	58	59	59
Retweet	24000	3	50	109	264
Conttime	9000	5	20	60	100
Stack Overflow	6633	22	41	72	736
Amazon	5200	16	14	45	94

Appendix J. Additional experiments

J.1. Visualisation for the synthetic data

We use the sparse SGCP to generate synthetic point process data, with the following settings: $m_1 = m_c = 50$, $T = 1$. The RBP kernel hyper-parameters in client 1 and client 2 are: $[1.5, 10]$, $[2, 8]$. The number of inducing points is 50. Fig. J.1 displays the traceplots of fitted random functions against that of groundtruth functions for two clients. Overall, our FedPP successfully captures almost all of the complex patterns presented in the groundtruth functions. In the top row, where the random functions $f_c(t)$ are compared, our fitted function closely aligns with the occurrences of events, even outperforming the groundtruth in certain places. For example, within the $[40, 50]$ interval allocated with dense events, the groundtruth function accidentally underestimates these events, whereas our FedPP reflects higher values. In the bottom row, our fitted intensity function is consistently lower than the groundtruth one, particularly during intervals with sparse events.

J.2. Main results on homogeneous setting

Apart from the heterogeneous setting, the homogeneous setting where local data across clients are IID is also crucial for testing the effectiveness of federated models. In TPPs tasks, it is natural to construct homogeneous data environments based on different types of events. In this experiment, we evaluate our proposed approach as well as other baselines on each type of events of all datasets. To ensure the validity and fairness of the results, for datasets Taobao, Stack Overflow, and Amazon, we individually select the top 5 event types with the largest sample sizes from each dataset. The numerical results are recorded in the following Tables J.1–J.5, from which we can see that our model FedPP shows comparable or better results on all datasets. Specifically, FedPP achieves the best results on three event types in the Taobao and two event types in the Stack Overflow and Amazon. For the Conttime dataset, FedPP only perform best in one of the five event types. Moreover, we find that Fed-SAHP, Fed-THP and Fed-Dec-ODE demonstrate strong competitiveness across all datasets.

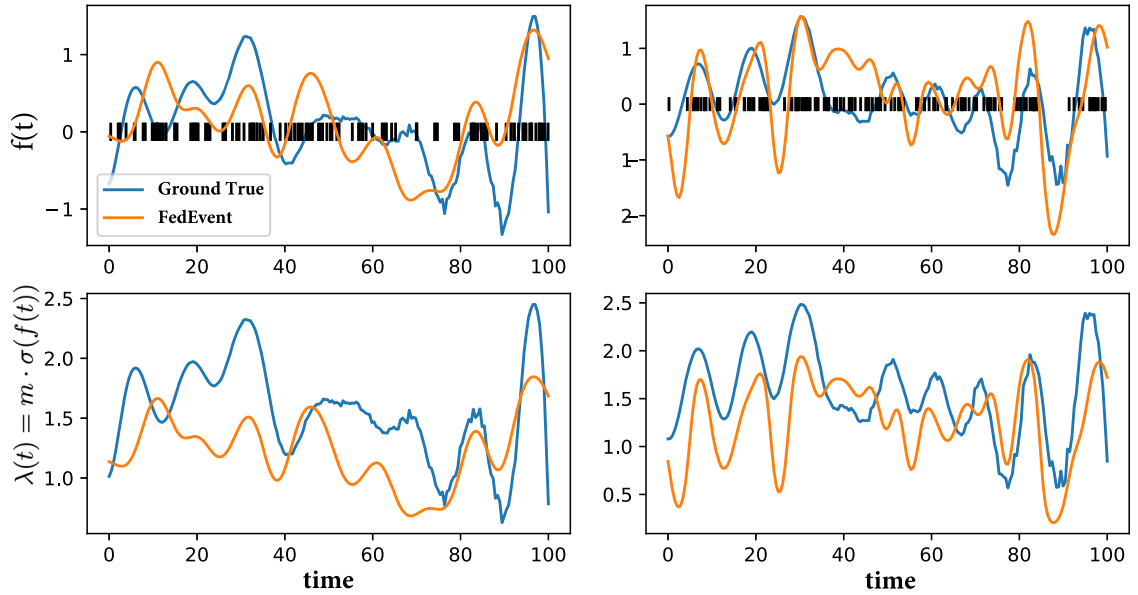


Fig. J.1. Visualisations of models for two clients in the synthetic data (Client 1 for the left panel, Client 2 for the right panel). Top row: Gaussian process random functions $\{f_c(t)\}_{c=1}^2$. Bottom row: intensity functions $\{m_c \cdot \sigma(f_c(t))\}_{c=1}^2$. The short black lines represent the generated events within the time interval.

Table J.1

The comparison of test log-likelihood under homogeneous settings on the Taobao dataset.

Taobao					
Types	Type 1	Type 2	Type 3	Type 4	Type 5
Fed-RMTPP	-2.45 ± 0.09	-1.34 ± 0.08	-1.22 ± 0.07	-2.49 ± 0.07	-1.80 ± 0.08
Fed-SAHP	-0.75 ± 0.02	-0.87 ± 0.02	-1.24 ± 0.11	-1.32 ± 0.09	-1.68 ± 0.03
Fed-THP	-0.80 ± 0.04	-0.85 ± 0.13	-1.35 ± 0.08	-1.07 ± 0.02	-1.72 ± 0.06
Fed-TCVAE	$\geq -1.23 \pm 0.07$	$\geq -1.45 \pm 0.01$	$\geq -1.55 \pm 0.06$	$\geq -2.03 \pm 0.12$	$\geq -2.04 \pm 0.09$
Fed-TCDDM	$\geq -1.67 \pm 0.05$	$\geq -1.48 \pm 0.03$	$\geq -1.28 \pm 0.03$	$\geq -2.07 \pm 0.08$	$\geq -1.83 \pm 0.04$
Fed-Dec-ODE	-0.78 ± 0.03	-0.96 ± 0.01	-1.30 ± 0.06	-0.97 ± 0.03	-1.74 ± 0.02
FedPP	-0.73 ± 0.01	-0.99 ± 0.03	-1.19 ± 0.01	-0.88 ± 0.01	-1.79 ± 0.03

Table J.2

The comparison of test log-likelihood under homogeneous settings on the Retweet datasets.

Retweet				
Types	Type 1	Type 2	Type 3	
Fed-RMTPP	-6.34 ± 0.07	-6.57 ± 0.05	-10.76 ± 0.11	
Fed-SAHP	-4.03 ± 0.01	-5.23 ± 0.02	-9.27 ± 0.04	
Fed-THP	-4.12 ± 0.06	-5.74 ± 0.04	-9.03 ± 0.09	
Fed-TCVAE	$\geq -4.27 \pm 0.08$	$\geq -6.74 \pm 0.12$	$\geq -10.58 \pm 0.13$	
Fed-TCDDM	$\geq -4.35 \pm 0.04$	$\geq -6.55 \pm 0.07$	$\geq -10.14 \pm 0.02$	
Fed-Dec-ODE	-3.97 ± 0.06	-5.88 ± 0.04	-9.13 ± 0.06	
FedPP	-3.27 ± 0.03	-6.35 ± 0.02	-10.03 ± 0.05	

Table J.3

The comparison of test log-likelihood under homogeneous settings on the Conttime dataset.

Conttime					
Types	Type 1	Type 2	Type 3	Type 4	Type 5
Fed-RMTPP	-2.77 ± 0.03	-2.67 ± 0.08	-3.01 ± 0.08	-3.22 ± 0.06	-2.66 ± 0.07
Fed-SAHP	-1.85 ± 0.04	-1.64 ± 0.05	-2.24 ± 0.02	-2.86 ± 0.04	-2.45 ± 0.03
Fed-THP	-1.96 ± 0.07	-1.77 ± 0.03	-2.77 ± 0.03	-2.13 ± 0.07	-2.73 ± 0.09
Fed-TCVAE	$\geq -3.02 \pm 0.01$	$\geq -2.82 \pm 0.07$	$\geq -3.36 \pm 0.09$	$\geq -3.77 \pm 0.15$	$\geq -3.05 \pm 0.06$
Fed-TCDDM	$\geq -2.83 \pm 0.08$	$\geq -2.93 \pm 0.06$	$\geq -2.98 \pm 0.12$	$\geq -3.69 \pm 0.09$	$\geq -3.72 \pm 0.04$
Fed-Dec-ODE	-1.87 ± 0.02	-1.58 ± 0.03	-2.68 ± 0.05	-2.44 ± 0.02	-2.53 ± 0.04
FedPP	-2.17 ± 0.08	-2.87 ± 0.09	-2.68 ± 0.06	-2.04 ± 0.09	-2.62 ± 0.04

Table J.4

The comparison of test log-likelihood under homogeneous settings on the Stack Overflow dataset.

Stack Overflow					
Types	Type 1	Type 2	Type 3	Type 4	Type 5
Fed-RMTPP	-2.88 ± 0.07	-3.14 ± 0.06	-2.97 ± 0.06	-1.99 ± 0.08	-2.44 ± 0.07
Fed-SAHP	-3.01 ± 0.02	-2.06 ± 0.01	-2.24 ± 0.01	-0.45 ± 0.02	-1.22 ± 0.03
Fed-THP	-2.97 ± 0.09	-2.13 ± 0.12	-2.37 ± 0.08	-0.33 ± 0.01	-1.97 ± 0.03
Fed-TCVAE	$\geq -3.45 \pm 0.05$	$\geq -3.55 \pm 0.15$	$\geq -3.02 \pm 0.15$	$\geq -2.94 \pm 0.12$	$\geq -2.53 \pm 0.17$
Fed-TCDDM	$\geq -3.67 \pm 0.06$	$\geq -2.87 \pm 0.09$	$\geq -2.96 \pm 0.11$	$\geq -1.93 \pm 0.07$	$\geq -2.33 \pm 0.10$
Fed-Dec-ODE	-2.28 ± 0.03	-2.08 ± 0.03	-2.17 ± 0.03	-0.42 ± 0.05	-1.88 ± 0.02
FedPP	-2.17 ± 0.09	-2.09 ± 0.01	-2.32 ± 0.02	-0.08 ± 0.01	-1.73 ± 0.02

Table J.5

The comparison of test log-likelihood under homogeneous settings on the Amazon dataset.

Amazon					
Types	Type 1	Type 2	Type 3	Type 4	Type 5
Fed-RMTPP	-1.27 ± 0.04	-1.33 ± 0.05	-3.16 ± 0.18	-1.23 ± 0.04	-1.72 ± 0.07
Fed-SAHP	-0.06 ± 0.03	0.59 ± 0.02	-0.02 ± 0.04	-0.79 ± 0.02	-0.62 ± 0.04
Fed-THP	0.75 ± 0.02	0.47 ± 0.08	-2.97 ± 0.09	-0.24 ± 0.07	-1.12 ± 0.13
Fed-TCVAE	$\geq -2.13 \pm 0.14$	$\geq -0.35 \pm 0.12$	$\geq -3.49 \pm 0.17$	$\geq -2.34 \pm 0.13$	$\geq -1.77 \pm 0.05$
Fed-TCDDM	$\geq -2.07 \pm 0.11$	$\geq -0.87 \pm 0.13$	$\geq -3.24 \pm 0.06$	$\geq -1.34 \pm 0.07$	$\geq -2.34 \pm 0.14$
Fed-Dec-ODE	-0.08 ± 0.06	0.33 ± 0.04	-0.15 ± 0.07	-0.22 ± 0.01	-0.57 ± 0.03
FedPP	1.06 ± 0.03	0.57 ± 0.16	-0.09 ± 0.07	-0.35 ± 0.03	-0.51 ± 0.01

References

- [1] O. Shchur, A.C. Türkmen, T. Januschowski, S. Günnemann, Neural temporal point processes: a review, [arXiv:2104.03528](#) (2021).
- [2] H. Lin, C. Tan, L. Wu, Z. Gao, S. Li, et al., An empirical study: extensive deep temporal point process, [arXiv:2110.09823](#) (2021).
- [3] Z. Wang, R. Jiang, H. Xue, F.D. Salim, X. Song, R. Shibasaki, W. Hu, S. Wang, Learning spatio-temporal dynamics on mobility networks for adaptation to open-world events, *Artif. Intell.* 335 (2024) 104120.
- [4] X. Li, Y. Gong, W. Liu, Y. Yin, Y. Zheng, L. Nie, Dual-track spatio-temporal learning for urban flow prediction with adaptive normalization, *Artif. Intell.* 328 (2024) 104065.
- [5] S.W. Linderman, Bayesian Methods for Discovering Structure in Neural Spike Trains, Ph.D. thesis, 2016.
- [6] I. Apostolopoulou, S. Linderman, K. Miller, A. Dubrawski, Mutually regressive point processes, *Adv. Neural Inf. Process. Syst.* 32 (2019) 5116–5127.
- [7] J. Yan, H. Xu, L. Li, Modeling and applications for temporal point processes, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 3227–3228.
- [8] J. Enguehard, D. Busbridge, A. Bozson, C. Woodcock, N. Hammerla, Neural temporal point processes for modelling electronic health records, in: *Machine Learning for Health*, PMLR, 2020, pp. 85–113.
- [9] A.G. Hawkes, Hawkes processes and their applications to finance: a review, *Quant. Finance* 18 (2) (2018) 193–198.
- [10] L. Fan, H. Yang, J. Zhai, X. Zhang, Forecasting stock volatility during the stock market crash period: the role of Hawkes process, *Finance Res. Lett.* 55 (2023) 103839.
- [11] N. Du, H. Dai, R. Trivedi, U. Upadhyay, M. Gomez-Rodriguez, L. Song, Recurrent marked temporal point processes: embedding event history to vector, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1555–1564.
- [12] M. Okawa, T. Iwata, T. Kurashima, Y. Tanaka, H. Toda, N. Ueda, Deep mixture point processes: spatio-temporal event prediction with rich contextual information, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 373–383.
- [13] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: *Artificial Intelligence and Statistics*, PMLR, 2017, pp. 1273–1282.
- [14] J. Konečný, H.B. McMahan, F.X. Yu, P. Richtárik, A.T. Suresh, D. Bacon, Federated learning: strategies for improving communication efficiency, [arXiv:1610.05492](#) (2016).
- [15] P. Hu, Z. Lin, W. Pan, Q. Yang, X. Peng, Z. Ming, Privacy-preserving graph convolution network for federated item recommendation, *Artif. Intell.* 324 (2023) 103996.
- [16] H. Du, C. Cheng, C. Ni, A unified momentum-based paradigm of decentralized SGD for non-convex models and heterogeneous data, *Artif. Intell.* 332 (2024) 104130.
- [17] C. Donner, M. Opper, Efficient Bayesian inference of sigmoidal Gaussian Cox processes, *J. Mach. Learn. Res.* 19 (1) (2018) 2710–2743.
- [18] F. Zhou, Z. Li, X. Fan, Y. Wang, A. Sowmya, F. Chen, Efficient inference for nonparametric Hawkes processes using auxiliary latent variables, *J. Mach. Learn. Res.* 21 (241) (2020) 1–31.
- [19] F. Zhou, Y. Zhang, J. Zhu, Efficient inference of flexible interaction in spiking-neuron networks, in: *International Conference on Learning Representations*, 2020.
- [20] S. Zuo, H. Jiang, Z. Li, T. Zhao, H. Zha, Transformer Hawkes process, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 11692–11702.
- [21] Q. Zhang, A. Lipani, O. Kirnap, E. Yilmaz, Self-attentive Hawkes process, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 11183–11193.
- [22] D.J. Daley, D. Vere-Jones, *An Introduction to the Theory of Point Processes: Volume II: General Theory and Structure*, Springer Science & Business Media, 2007.
- [23] S. Xue, X. Shi, Z. Chu, Y. Wang, H. Hao, F. Zhou, C. JIANG, C. Pan, J.Y. Zhang, Q. Wen, et al., EasyTPP: towards open benchmarking temporal point processes, in: *The Twelfth International Conference on Learning Representations*, 2023.
- [24] L. Cao, Beyond I.I.D.: non-IID thinking, informatics, and learning, *IEEE Intell. Syst.* 37 (4) (2022) 5–17.
- [25] H. Chen, H. Liu, Z. Wu, X. Fan, L. Cao, FedSI: federated subnetwork inference for efficient uncertainty quantification, [arXiv:2404.15657](#) (2024).
- [26] X. Fan, B. Li, F. Zhou, S. Sisson, Continuous-time edge modelling using non-parametric point processes, in: *Advances in Neural Information Processing Systems*, 2021, pp. 2319–2330.
- [27] F. Zhou, Q. Kong, Z. Deng, F. He, P. Cui, J. Zhu, Heterogeneous multi-task Gaussian Cox processes, *Mach. Learn.* 112 (12) (2023) 5105–5134.
- [28] C.M. Bishop, *Pattern Recognition and Machine Learning* (Information Science and Statistics), Springer-Verlag, 2006.
- [29] H. Chen, H. Liu, L. Cao, T. Zhang, Bayesian personalized federated learning with shared and personalized uncertainty representations, [arXiv:2309.15499](#) (2023).
- [30] M. Patacchiola, J. Turner, E.J. Crowley, M. O’Boyle, A.J. Storkey, Bayesian meta-learning for the few-shot setting via deep kernels, *Adv. Neural Inf. Process. Syst.* 33 (2020) 16108–16118.
- [31] M.K. Titsias, F.J.R. Ruiz, S. Nikoloutsopoulos, A. Galashov, Information theoretic meta learning with Gaussian processes, in: *Uncertainty in Artificial Intelligence*, PMLR, 2021, pp. 1597–1606.
- [32] H. Yu, K. Guo, M. Karami, X. Chen, G. Zhang, P. Poupart, Federated Bayesian neural regression: a scalable global federated Gaussian process, [arXiv:2206.06357](#) (2022).
- [33] I. Achituve, A. Shamsian, A. Navon, G. Chechik, E. Fetaya, Personalized federated learning with Gaussian processes, *Adv. Neural Inf. Process. Syst.* 34 (2021) 8392–8406.
- [34] Z. Wang, Z. Miao, X. Zhen, Q. Qiu, Learning to learn dense Gaussian processes for few-shot learning, *Adv. Neural Inf. Process. Syst.* 34 (2021) 13230–13241.
- [35] X. Zhang, Y. Li, W. Li, K. Guo, Y. Shao, Personalized federated learning via variational Bayesian inference, in: *International Conference on Machine Learning*, PMLR, 2022, pp. 26293–26310.
- [36] P. Reynaud-Bouret, V. Rivoirard, C. Tuleau-Malot, Inference of functional connectivity in neurosciences via Hawkes processes, in: *2013 IEEE Global Conference on Signal and Information Processing*, IEEE, 2013, pp. 317–320.
- [37] F. Gerhard, M. Deger, W. Truccolo, On the stability and dynamics of stochastic spiking neuron models: nonlinear Hawkes process and point process GLMs, *PLoS Comput. Biol.* 13 (2) (2017) e1005390.
- [38] H. Mei, J.M. Eisner, The neural Hawkes process: a neurally self-modulating multivariate point process, *Adv. Neural Inf. Process. Syst.* 30 (2017) 6754–6764.
- [39] H. Lin, L. Wu, G. Zhao, L. Pai, S.Z. Li, Exploring generative neural temporal point process, *Trans. Mach. Learn. Res.* (2022).
- [40] M. Garnelo, J. Schwarz, D. Rosenbaum, F. Viola, D.J. Rezende, S.M. Eslami, Y.W. Teh, Neural processes, [arXiv:1807.01622](#) (2018).
- [41] M. Garnelo, D. Rosenbaum, C. Maddison, T. Ramalho, D. Saxton, M. Shanahan, Y.W. Teh, D. Rezende, S.M.A. Eslami, Conditional neural processes, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 1704–1713.
- [42] G. Singh, J. Yoon, Y. Son, S. Ahn, Sequential neural processes, *Adv. Neural Inf. Process. Syst.* 32 (2019) 10254–10264.
- [43] H. Huang, H. Wang, B. Mak, Recurrent poisson process unit for speech recognition, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 33, 2019, pp. 6538–6545.
- [44] H. Wang, D.-Y. Yeung, A survey on Bayesian deep learning, *ACM Comput. Surv.* 53 (5) (2020) 1–37.
- [45] Y. Song, L. Donghyun, R. Meng, W.H. Kim, Decoupled marked temporal point process using neural ordinary differential equations, in: *The Twelfth International Conference on Learning Representations*, 2024.
- [46] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, [arXiv:1412.6980](#) (2014).
- [47] J. Xu, M. Yang, W. Ding, S.-L. Huang, Stabilizing and improving federated learning with highly non-iid data and client dropout, *Appl. Intell.* 55 (3) (2025) 216.
- [48] S.P. Jenkins, Survival analysis, Unpublished Manuscript Inst. Soc. Econ. Res. Univ. Essex Colchester UK 42 (2005) 54–56.
- [49] P. Wang, Y. Li, C.K. Reddy, Machine learning for survival analysis: a survey, *ACM Comput. Surv.* 51 (6) (2019) 1–36.
- [50] S. Rahimian, R. Kerkouche, I. Kurth, M. Fritz, Practical challenges in differentially-private federated survival analysis of medical data, in: *Conference on Health, Inference, and Learning*, PMLR, 2022, pp. 411–425.
- [51] M.M. Rahman, S. Purushotham, FedPseudo: privacy-preserving pseudo value-based deep learning models for federated survival analysis, in: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 1999–2009.

- [52] M.M. Rahman, S. Purushotham, Communication-efficient pseudo value-based random forests for federated survival analysis, in: *Proceedings of the AAAI Symposium Series*, 2, 2023, pp. 458–466.
- [53] A. Damianou, N.D. Lawrence, Deep Gaussian processes, in: *Artificial Intelligence and Statistics*, PMLR, 2013, pp. 207–215.
- [54] A.G. Wilson, Z. Hu, R. Salakhutdinov, E.P. Xing, Deep kernel learning, in: *Artificial Intelligence and Statistics*, PMLR, 2016, pp. 370–378.
- [55] P. Batz, A. Ruttor, M. Opper, Approximate bayes learning of stochastic differential equations, *Phys. Rev. E* 98 (2) (2018) 022109.
- [56] C. Donner, M. Opper, Efficient Bayesian inference for a Gaussian process density model, [arXiv:1805.11494](https://arxiv.org/abs/1805.11494) (2018).